

Codebook of UltraGrammer

Contents

Contents

1 你是否有...

1.1 嘗試...

1.2 檢查...

2 資料結構

2.1 BIT (Fenwick Tree)

2.2 Sparse Table

2.3 Segment Tree 線段樹

2.4 Prefix Sum(前綴和) 和 Difference(差分)

2.5 Monotonic Stack 單調棧

2.6 Monotonic Queue 單調隊列

3 數論

3.1 快速冪

3.2 模逆元與費馬小定理

3.3 組合數 C(n,k)

3.4 質數檢查

3.5 大數乘法

3.6 排容原理

3.7 擴展歐幾里得 (Extended GCD)

3.8 尤拉函數

3.9 數論分塊

4 字串

4.1 KMP 字串匹配

4.2 Z-Algorithm 字串匹配

4.3 Manacher 迴文判斷

4.4 Trie 字典樹

4.5 AC 自動機

5 圖論

5.1 BFS 與拓樸排序

5.2 DFS

5.3 Dijkstra 單源最短路徑

5.4 SPFA 單源最短路徑

5.5 Floyd-Warshall 多源最短路徑

5.6 二分圖判定 (著色問題)

5.7 割點與雙連通分量

5.8 匈牙利演算法

5.9 KM 演算法

5.10 Dinic 最大流

6 幾何

6.1 點與向量基礎 (Point & Vector)

6.2 判斷線段、矩形相交 (Segment Intersection)

6.3 點與線段關係 (Point & Line)

6.4 任意多邊形面積

6.5 凸包 Convex Hull

7 樹論

7.1 Disjoint Set

7.2 Kruskal 最小生成樹

7.3 次小生成樹

7.4 最近共同祖先 LCA

8 DP

8.1 01 Knapsack 背包

8.2 LCS 最長共同子序列 (Sequence)

8.3 LIS / LDS 最長遞增/遞減子序列

8.4 LCS 轉 LIS

8.5 區間 DP 概念 (最長迴文子序列)

8.6 跳格子

8.7 分錢幣問題

8.8 Kadane's Algorithm (最大子陣列和)

9 Greedy

9.1 一維區間全覆蓋

9.2 間格跳躍

9.3 最大不重疊區間

9.4 最小覆蓋區間

9.5 區間選點問題

9.6 最小化最大延遲問題

9.7 Huffman 樹

9.8 任務調度問題

9.9 多機調度問題

9.10 最大區間重疊數 Sweep Line

9.11 Josephus 約瑟夫問題

10 Search & Techniques

10.1 Binary Search

10.2 Ternary Search

10.3 Two Pointer 雙指針

11 MISC

11.1 模板

11.2 一些概念

11.3 自訂 Hash Function

11.4 C++ 函式庫

11.5 對拍器

1 你是否有...

1.1 嘗試...

1.2 檢查...

1.1 嘗試...

1.2 檢查...

2 資料結構

2.1 BIT (Fenwick Tree)

用法: 支援單點修改與區間求和。必須是 1-based index。

時間複雜度: $\mathcal{O}(\log n)$

```
1 struct BIT {
2     int n;
3     vector<long long> tree; // 使用 11 避免區間總和溢位
4
5     void init(int sz){
6         n = sz;
7         tree.assign(sz+1, 0);
8     }
9
10    inline int lowbit(int x) { // 取得二進位中最右邊的 1
11        return x & (-x);
12    }
13
14    void add(int x, long long val) { // 在位置 x 加上了 val
15        for (; x <= n; x += lowbit(x)) {
```

```

16         tree[x] += val;
17     }
18 }
19
20 long long query(int x) { // 計算 [1, x] 的總和
21     long long sum = 0;
22     for (; x > 0; x -= lowbit(x)) {
23         sum += tree[x];
24     }
25     return sum;
26 }
27
28 long long query(int l, int r) { // 計算 [l, r] 的總和 (前綴和相減)
29     return query(r) - query(l - 1);
30 }
31 };

```

2.2 Sparse Table

用法: 完全只為區間 (最大最小值) 查詢

時間複雜度: 建表 $\mathcal{O}(\log n)$ 查詢 $\mathcal{O}(1)$

```

1  const int MAXN = 200005; // 依 N 大小變動
2  const int LOG = 20; // LOG = floor(ln(N))+1, 在 N=10^6 範圍時, 20 就夠了
3  int st[MAXN][LOG];
4
5  // 建表 (以區間最大值為例)
6  void build_ST(int n, const vector<int>& a) {
7      for(int i = 0; i < n; i++) st[i][0] = a[i];
8      for(int j = 1; (1 <= j) <= n; j++)
9          for(int i = 0; i + (1 <= j) - 1 < n; i++)
10             st[i][j] = max(st[i][j-1], st[i + (1 <= j) - 1][j-1]);
11 }
12
13 // 查詢 [L, R]
14 int query(int L, int R) {
15     int j = __lg(R - L + 1);
16     return max(st[L][j], st[R - (1 <= j) + 1][j]);
17 }

```

2.3 Segment Tree 線段樹

用法: 給定 n 筆資料查詢與 m 次範圍更新, 解決多次範圍更新的問題, 找”區間和”或最大/小值。必須是 1-based index。

時間複雜度: 建樹 $\mathcal{O}(N)$ 查詢/更新 $\mathcal{O}(\log N)$

```

1  using ll = long long;
2
3  vector<ll> arr, sum/tree, add; //宣告成全域, arr 是原始數據 (輸入), sum/tree
   是範圍累加和/區間查詢, add 是 lazy tag 儲存的值
4
5  void up(int i){ // range_sum +=, max = max, min = min, gcd = __gcd
6      sum[i] = sum[i*2] + sum[i*2+1];
7  }
8
9  void lazy(int i, ll v, int n){ // 懶標記 (lazy tag), 暫存當前覆蓋範圍
10     sum[i] += v*n; // 如果不是累加和, 改成 tree[i] += v;
11     add[i] += v;
12 }
13
14 void down(int i, int ln, int rn){ // 往下分發 lazy tag
15     if(add[i] != 0){ // 0 或 INF 或 -INF, 依線段樹邏輯選擇
16         lazy(i*2, add[i], ln);
17         lazy(i*2+1, add[i], rn);
18         add[i] = 0; // 0 或 INF 或 -INF, 依線段樹邏輯選擇
19     }
20 }
21
22 void build(int l, int r, int i){ // 遞迴式初始化 (init)
23     if(l == r) sum[i] = arr[l]; // 如果區間只剩一個數值, 就直接賦值
24     else{
25         int mid = l + (r-l)/2; // 找中點
26         build(l, mid, i*2); // 左半邊 build
27         build(mid+1, r, i*2+1); // 右半邊 build
28         up(i); // 自己 = 左 + 右 區間和
29     }
30     add[i] = 0; // 0 或 INF 或 -INF, 依線段樹邏輯選擇
31 }
32
33 void update(int jobl, int jobr, ll jobv, int l, int r, int i){ // 更新區間數
   值 (jobl ~ jobr 加上 jobv), 用 l, r, i 判斷範圍
34     if(jobl <= l && r <= jobr) lazy(i, jobv, r-l+1); // 如果範圍被包的話就直接
   lazy
35     else{
36         int mid = l + (r-l)/2;
37         down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
38         if(jobl <= mid) update(jobl, jobr, jobv, l, mid, i*2); // 判斷左右區
   段是否需要更新
39         if(jobr > mid) update(jobl, jobr, jobv, mid+1, r, i*2+1);
40         up(i); // 最後往上回傳
41     }
42 }
43
44 ll query(int jobl, int jobr, int l, int r, int i){
45     if(jobl <= l && r <= jobr) return sum[i]; // 如果包範圍就直接 return 回去
46
47     int mid = l + (r-l)/2;
48     down(i, mid-l+1, r-mid); // 記得往下分發 lazy tag
49     ll total = 0; // range sum / gcd = 0, max = -INF, min = INF
50     if(jobl <= mid) total += query(jobl, jobr, l, mid, i*2); // range_sum =
   +=, max = max, min = min, gcd = __gcd
51     if(jobr > mid) total += query(jobl, jobr, mid+1, r, i*2+1);
52     return total; // 因為只是 query, 沒有修改, 所以不用 up 更新回去
53 }
54
55 int main() {
56     int n, m;
57     cin >> n >> m;
58     arr.assign(n+1, 0);

```

```

59     sum.assign(4*(n+1), 0); // 大小必須開到 4 倍 n+1
60     add.assign(4*(n+1), 0);
61
62
63     for(int i=1; i<=n; ++i) cin >> arr[i];
64     build(1, n, 1);
65     int op;
66     ll x, y, k;
67     while(m--){
68         cin >> op;
69         if(op == 1){
70             cin >> x >> y >> k;
71             update(x, y, k, 1, n, 1);
72         }
73         else{
74             cin >> x >> y;
75             cout << query(x, y, 1, n, 1) << '\n';
76         }
77     }
78     return 0;
79 }

```

2.4 Prefix Sum(前綴和) 和 Difference(差分)

用法: Prefix sum: 多次區間查詢/區間和, Difference: 大量區間更新

時間複雜度: 建表 $\mathcal{O}(N)$, 查詢 $\mathcal{O}(1)$

```

1  using ll = long long;
2
3  // === 1D prefix ===
4  // 建前綴表 build
5  for (int i = 1; i <= n; i++)
6      pre[i] = pre[i-1] + a[i];
7
8  // 如果是 XOR 就把 + 換 ^ -> pre[i] = pre[i-1] ^ a[i];
9
10 // 查詢含 l,r 的區間 query [l, r] = query [0, r] - query [0, l-1]
11 ll query(int l, int r) {
12     return pre[r] - pre[l-1];
13     // XOR(l,r) = pre[r] ^ pre[l-1];
14 }
15
16 // === 2D prefix ===
17
18 // pre 是前綴和 array, a 是原始數據 array
19 // 必須全是 "1-indexed", 避免 [-1] 的判斷
20 // i 是 row, 從 1~n, j 是 col, 從 1~m
21 void build() {
22     // 迴圈從 1 開始, 當 i=1 或 j=1 時, 存取到 0 的位置都會剛好是 0, 完美相消
23     for (int i = 1; i <= n; ++i) {
24         for (int j = 1; j <= m; ++j) {
25             pre[i][j] = pre[i-1][j] + pre[i][j-1] - pre[i-1][j-1] + a[i][j];
26         }
27     }
28 }
29
30 // query[(x1, y1) - (x2, y2)], 傳入的 x1, y1, x2, y2 也必須是 1-based 的索引
31 int query(int x1, int y1, int x2, int y2) {
32     return pre[x2][y2] - pre[x1-1][y2] - pre[x2][y1-1] + pre[x1-1][y1-1];
33 }
34
35 // === 1D Difference ===
36
37 // 區間加 val -> 從 l + val, 到 r+1 不再加 val
38 diff[l] += val;
39 diff[r+1] -= val;
40
41 // 還原值
42 for (int i = 1; i <= n; i++)
43     a[i] = a[i-1] + diff[i];
44
45 // === 2D Difference ===
46
47 // 區間加 k
48 diff[x1][y1] += k;
49 diff[x2+1][y1] -= k;
50 diff[x1][y2+1] -= k;
51 diff[x2+1][y2+1] += k;
52
53 // 還原
54 for (int i = 1; i <= n; i++) {
55     for (int j = 1; j <= m; j++) {
56         diff[i][j] += diff[i-1][j]
57             + diff[i][j-1]
58             - diff[i-1][j-1];
59     }
60 }
61 // 此時 diff[i][j] 是 "經過處理後的值", 如果有原始陣列 a, 需再加上 a[i][j]

```

2.5 Monotonic Stack 單調棧

用法: 找類似 Next Greater Element 這樣的題目

時間複雜度: $\mathcal{O}(N)$

```

1  // === Next Greater Element ===
2  int a[105] = {}; // 存好值的 array
3  int ans[105] = {}; // 存答案的 array
4  stack<int> st; // 注意是存 index 不是 value, value 在 a[st.top()]
5
6  for (int i = 0; i < n; i++) {
7      // 第一輪會跳過, 因為 st 是 empty
8      while (!st.empty() && a[st.top()] < a[i]) {
9          ans[st.top()] = i;
10         st.pop();
11     }
12     st.push(i);
13 }
14
15 while (!st.empty()) { // 重設

```

```

17     ans[st.top()] = -1;
18     st.pop();
19 }
20
21 // === Next Smaller Element ===
22 只需要把 while 改成
23 while (!st.empty() && a[st.top()] > a[i]) // (因為要找更小)
24
25 // === Previous Greater ===
26 for (int i = 0; i < n; i++) {
27     while (!st.empty() && a[st.top()] <= a[i])
28         st.pop();
29
30     if (st.empty())
31         ans[i] = -1;
32     else
33         ans[i] = st.top();
34
35     st.push(i);
36 }
37 // 最後 ans 陣列就是答案

```

2.6 Monotonic Queue 單調隊列

用法: 直接用來找 Sliding Window Maximum/Minimum, 使用 deque
時間複雜度: $\mathcal{O}(N)$

```

1 deque<int> dq; // 注意一樣是存 index
2 int k = 5; // 最大窗口大小 maximum window length
3
4 // 以尋找最長遞減序列為例 (maximum)
5 for (int i = 0; i < n; i++) {
6     // pop 掉超出窗口的部分
7     while (!dq.empty() && dq.front() <= i - k)
8         dq.pop_front();
9
10    // 維持遞減狀態
11    while (!dq.empty() && a[dq.back()] <= a[i])
12        dq.pop_back();
13    // 如果是 minimum 的話 改成
14    while (!dq.empty() && a[dq.back()] >= a[i])
15
16
17    dq.push_back(i);
18
19    if (i >= k - 1)
20        ans.push_back(a[dq.front()]);
21 }

```

3 數論

3.1 快速冪

用法: 計算 $a^b \pmod m$ 。

時間複雜度: $\mathcal{O}(\log b)$

```

1 // 計算 (a^b) % mod, 值常為 1e9+7 或 998244353
2 long long fast_pow(long long a, long long b, long long mod) {
3     long long res = 1;
4     a %= mod; // 預防初始底數 a 大於等於 mod
5     while (b > 0) {
6         // 如果當前次方數是奇數, 把當前的 a 乘進答案裡
7         if (b & 1) res = res * a % mod;
8         a = a * a % mod; // 底數平方 % mod
9         b >>= 1; // 次方數除以 2
10    }
11    return res;
12 }

```

3.2 模逆元與費馬小定理

用法: p 必須是質數, 且整數 a 與 p 互質, 則 $a^{p-1} \equiv 1 \pmod p$ 。

兩邊同除以 a 可得 $a^{p-2} \equiv a^{-1} \pmod p$ 。即計算 $(x/a) \pmod p$ 等同於計算 $(x \cdot a^{p-2}) \pmod p$ 。

時間複雜度: $\mathcal{O}(\log p)$

```

1 long long inv(long long a, long long mod) {
2     return fast_pow(a, mod - 2, mod);
3 }
4
5 // 應用範例: 計算 (a / b) % mod
6 long long mod_div(long long a, long long b, long long mod) {
7     return (a % mod) * inv(b, mod) % mod;
8 }

```

3.3 組合數 $C(n, k)$

用法: $N \leq 2000$ 用帕斯卡三角形。 $N \leq 10^6$ 且模數為質數時, 用預處理階乘與模逆元

時間複雜度: $\mathcal{O}(N^2)$ 或 $\mathcal{O}(N)$

```

1 const long long MOD = 1e9 + 7; // 依照題目給的質數修改
2
3 // 一、帕斯卡三角形 (N <= 2000), 全加法無除法, 不怕 MOD 非質數,  $\mathcal{O}(N^2)$ 
4 const int MAXN = 2005;
5 long long C[MAXN][MAXN];
6
7 void build_pascal() {
8     for (int i = 0; i < MAXN; ++i) {
9         C[i][0] = 1; // 從 n 個取 0 個的方法數永遠是 1
10        for (int j = 1; j <= i; ++j) C[i][j] = (C[i-1][j-1] + C[i-1][j]) % MOD;
11    }
12 }
13
14 // 二、階乘與模逆元 (N <= 10^6), MOD 必須是質數,  $\mathcal{O}(N)$ 

```

```

15 long long fact[MAXN], invFact[MAXN];
16
17 void build_fact() { // 建表
18     fact[0] = invFact[0] = 1;
19     for (int i = 1; i < MAXN; ++i) fact[i] = fact[i-1] * i % MOD;
20     invFact[MAXN-1] = fast_pow(fact[MAXN-1], MOD-2); // 使用快速冪
21     for (int i = MAXN-2; i >= 1; --i) invFact[i] = invFact[i+1] * (i+1) % MOD;
22 }
23
24 long long nCr(int n, int k) { // 查詢
25     if (k < 0 || k > n) return 0;
26     return fact[n] * invFact[k] % MOD * invFact[n-k] % MOD;
27 }

```

3.4 質數檢查

用法: 單次詢問大數用 isPrime; 若有大量詢問則建質數表。

時間複雜度: $\mathcal{O}(\sqrt{N})$

```

1 // 一、直接查, 只問極少次, 且數字可能很大 (n<= 1e14)
2 bool isPrime(long long n) {
3     if (n < 2) return false;
4     if (n == 2 || n == 3) return true;
5     if (n % 2 == 0 || n % 3 == 0) return false;
6
7     for (long long i = 5; i * i <= n; i += 6) {
8         if (n % i == 0 || n % (i + 2) == 0) return false;
9     }
10    return true;
11 }
12
13 // 二、埃式篩, 建質數表, 大量查詢 1 ~ 10^7 內的質數
14 const int MAXN = 1e7 + 10;
15 vector<int> primes;
16 bool is_prime[MAXN];
17
18 void build(int n) {
19     primes.clear(); // 初始化
20     fill(is_prime, is_prime + n + 1, true); // 初始化全設為 true
21     is_prime[0] = is_prime[1] = false;
22
23     for (int i = 2; i <= n; ++i) {
24         if (is_prime[i]) {
25             primes.push_back(i); // 放到 primes vector 中記錄
26
27             for (ll j = (ll)i * i; j <= n; j += i) is_prime[j] = false;
28         }
29     }
30 }
31
32 // 三、歐拉篩
33 #define MAXN 47000 //sqrt(2^31)=46,340...
34 bool isPrime[MAXN];
35 int p[MAXN];
36 int pSize=0;
37 void getPrimes(){
38     pSize = 0; // 初始化
39     memset(isPrime, true, sizeof(isPrime));
40     isPrime[0] = isPrime[1] = false;
41     for(int i=2; i<MAXN; i++){
42         if(isPrime[i]) p[pSize++] = i;
43         for(int j=0; j<pSize && i*p[j] <= MAXN; ++j){
44             isPrime[i*p[j]] = false;
45             if(i%p[j]==0) break;
46         }
47     }
48 }
49
50
51 /*
52 problem :
53 給定整數 N, 求 N 最少可以拆成多少個質數的和。
54 如果 N 是質數, 則答案為 1。
55 如果 N 是偶數 (N!=2), 則答案為 2(強歌德巴赫猜想)。
56 如果 N 是奇數且 N-2 是質數, 則答案為 2(2+ 質數)。
57 其他狀況答案為 3(弱歌德巴赫猜想)。
58 */
59 bool isPrime(int n){
60     for(int i=2;i<n;++i){
61         if(i*i>n) return true;
62         if(n%i==0) return false;
63     }
64     return true;
65 }
66
67 int main(){
68     int n;
69     cin>>n;
70     if(isPrime(n)) cout<<"1\n";
71     else if(n%2==0||isPrime(n-2)) cout<<"2\n";
72     else cout<<"3\n";
73 }

```

3.5 大數乘法

用法: 未知长度的大數相乘

時間複雜度: $\mathcal{O}(N \times M)$

Range	擁有最多因數的數字	其因數總個數
10^9	735134400	1344
2^{31}	2095133040	1600
10^{18}	897612484786617600	103680
2^{64}	9200527969062830400	161280

```

1 // 一、BigInt * Int, 算陪乘、連續乘上範圍在 int 內的數值, 0(N)
2 vector<int> multiply_small(const vector<int>& a, int b) {
3     if (b == 0 || (a.size() == 1 && a[0] == 0)) return {0};
4     vector<int> c;
5     long long carry = 0; // 要用 ll 才不會溢位
6     for (size_t i = 0; i < a.size() || carry > 0; i++) {
7         if (i < a.size()) carry += 1LL * a[i] * b;
8         c.push_back(carry % 10);
9         carry /= 10;
10    }
11    return c;
12 }
13
14 // 二、BigInt * BigInt, 兩個長度未知的超大整數相乘, 0(N * M)
15 vector<int> multiply_big(const vector<int>& a, const vector<int>& b) {
16     if (a.empty() || b.empty()) return {0};
17     if ((a.size() == 1 && a[0] == 0) || (b.size() == 1 && b[0] == 0)) return {0};
18
19     vector<int> c(a.size() + b.size(), 0); // 結果的最大可能長度為兩者長度相加
20
21     for (size_t i = 0; i < a.size(); i++) {
22         long long carry = 0;
23         for (size_t j = 0; j < b.size() || carry > 0; j++) {
24             long long cur = c[i + j] + 1LL * a[i] * (j < b.size() ? b[j] : 0)
25             + carry;
26             c[i + j] = cur % 10;
27             carry = cur / 10;
28         }
29
30         // 移除多餘的前導零 (例如 00 變成 0)
31         while (c.size() > 1 && c.back() == 0) c.pop_back();
32
33         return c;
34 }
35
36 // 輔助函式：將字串轉換為大數陣列 (反向存入)
37 vector<int> string_to_bigint(const string& s) {
38     vector<int> res;
39     for (int i = s.length() - 1; i >= 0; i--) res.push_back(s[i] - '0');
40     return res;
41 }
42
43 // 輔助函式：印出大數 (反向印出)
44 void print_bigint(const vector<int>& a) {
45     for (int i = a.size() - 1; i >= 0; i--) cout << a[i];
46     cout << "\n";
47 }

```

3.6 排容原理

用法: 解決「至少符合一項條件」的計數問題。遵守奇加偶減原則。

時間複雜度: $\mathcal{O}(2^M \cdot M)$

```

1 // 題型：計算 1 ~ N 中，有幾個數字能被 primes 陣列中的「至少一個」質數整除
2 long long inclusion_exclusion(long long n, const vector<long long>& primes) {
3     int m = primes.size();
4     long long ans = 0;
5
6     // 利用 Bitmask 枚舉所有可能的子集組合 (1 到 2^m - 1)
7     // 例如 m=3, mask 從 001 數到 111
8     for (int mask = 1; mask < (1 << m); ++mask) {
9         long long lcm_val = 1; // 該子集的最小公倍數 (lcm)
10        int bits = 0; // 紀錄這個子集選了幾個質數 (奇/偶數)
11
12        for (int i = 0; i < m; ++i) {
13            // 檢查 mask 的第 i 位是否為 1 (代表選了第 i 個質數)
14            if ((mask >> i) & 1) {
15                bits++;
16                lcm_val *= primes[i]; // 質數的 LCM 直接相乘即可
17                if (lcm_val > n) break; // 防溢位與無效計算優化
18            }
19        }
20
21        if (lcm_val > n) continue; // 組合出來的條件比 N 還大，裡面絕對沒有倍數
22
23        long long count = n / lcm_val; // 滿足此條件的數字個數
24
25        if (bits % 2 == 1) ans += count; // 奇加
26        else ans -= count; // 偶減
27    }
28    return ans;
29 }
30
31 /* 變體應用：若題目問「1 ~ N 中與 K 互質的數有幾個？」
32 解法：先把 K 質因數分解，得到質數陣列 primes。
33 然後用 n 減去 inclusion_exclusion(n, primes) 即為答案。 */

```

3.7 擴展歐幾里得 (Extended GCD)

用法: 求解 $ax + by = \gcd(a, b)$, 以及當模數 m 非質數時的萬能模逆元。

時間複雜度: $\mathcal{O}(\log(\min(a, b)))$

```

1 /*
2 應用場景：
3 1. 題目要求解  $ax + by = \gcd(a, b)$ 
4 2. 題目要求你算「模逆元 (Modular Inverse)」，但是模數 p 不是質數。
5
6 對於  $ax + by = c$ ，若  $\gcd(a, b)$  不能整除  $c$  ( $c$  不是他的倍數)，就代表沒有整數解
7 否則，設  $g = \gcd(a, b)$ ，先解出基本解  $x'$ ,  $y'$ 
8 */
9 long long extgcd(long long a, long long b, long long &x, long long &y) {
10     if (b == 0) {
11         x = 1; y = 0;
12         return a;
13     }
14     long long x1, y1;
15     long long d = extgcd(b, a % b, x1, y1);
16     x = y1;

```

```

17     y = x1 - y1 * (a / b);
18     return d;
19 }
20
21 // 萬能模逆元：求 a 在 mod m 之下的逆元 (無論 m 是否為質數皆可)，-1 表示不存在
22 long long inverse(long long a, long long m) {
23     long long x, y;
24     long long g = extgcd(a, m, x, y);
25     if (g != 1) return -1; // 不互質，無解
26     return (x % m + m) % m; // 確保回傳正數
27 }

```

3.8 尤拉函數

用法: 求 $1 \sim N$ 中與 N 互質的整數個數。

時間複雜度: $\mathcal{O}(\sqrt{N})$

```

1 // 應用場景：題目問你「在 1 ~ N 裡面，有幾個數字跟 N 互質？」。
2 long long phi(long long n) {
3     long long res = n;
4     for (long long i = 2; i * i <= n; i++) {
5         if (n % i == 0) {
6             // 發現質因數 i，套用公式  $res = res * (1 - 1/i)$ 
7             while (n % i == 0) n /= i;
8             res -= res / i;
9         }
10    }
11    // 如果 n 本身最後剩下一個大於 sqrt(n) 的質數
12    if (n > 1) res -= res / n;
13    return res;
14 }

```

3.9 數論分塊

用法: 快速計算 $\sum \lfloor \frac{N}{i} \rfloor$ 。利用商的階梯狀分布，將 $\mathcal{O}(N)$ 降至 $\mathcal{O}(\sqrt{N})$ 。

時間複雜度: $\mathcal{O}(\sqrt{N})$

```

1 // 應用場景：題目要你算從 1 ~ N 中  $N / i$  取下高斯後的總和分數， $sum = sum($ 
2  $\text{floor}(n / i))$  for  $i = 1$  to  $n$ 
3 long long math_block(long long n) {
4     long long ans = 0;
5     // l 是當前區塊的左邊界，r 是右邊界
6     for (long long l = 1, r; l <= n; l = r + 1) {
7         r = n / (n / l); // 魔法公式：直接算出與 l 擁有相同 floor(n/i) 值的右邊界
8
9         // r - l + 1 是這個區塊的數字個數，(n / l) 是這個區塊共同的商
10        ans += (r - l + 1) * (n / l);
11    }
12    return ans;

```

4 字串

4.1 KMP 字串匹配

用法: 尋找 pattern 出現位置。pi 陣列可求最小循環節。回傳 0-based 起始位置，未匹配為 -1。pi[i] 代表前綴 $S[0 \dots i]$ 中最長的「相等真前後綴」長度。

時間複雜度: $\mathcal{O}(N + M)$

```

1 // pi 陣列 (Longest Prefix Suffix): pi[i] 代表 pattern[0..i] 的最長相同前後綴
2 // 長度
3 vector<int> build_pi(const string& s) {
4     int m = s.length();
5     vector<int> pi(m, -1);
6     for (int i = 1, j = -1; i < m; ++i) {
7         // 如果不匹配，就利用 pi 陣列往回跳
8         while (j >= 0 && s[i] != s[j+1]) j = pi[j];
9         if (s[i] == s[j+1]) j++;
10        pi[i] = j;
11    }
12    return pi;
13 }
14
15 // 可用 pi 陣列尋找一個字串的最小循環節
16 int getMinimumPeriod(const string& s) {
17     int n = s.length();
18     if (n == 0) return 0;
19     vector<int> pi = build_pi(s); // 取得 index 陣列
20
21     // 取得整個字串的最長相同前後綴長度
22     int maxPrefixLength = pi[n - 1] + 1; // 若 pi[n-1] == -1，則長度為 0
23
24     // 計算可能的最小循環節長度
25     int period = n - maxPrefixLength;
26
27     // 檢查總長度是否能被 period 整除
28     if (maxPrefixLength > 0 && n % period == 0) {
29         return period;
30     }
31
32     // 若無法整除，代表沒有更小的週期，最小循環節為字串本身
33     return n;
34 }
35
36 // 最終，最小循環節長度即為回傳的 period -> s.substr(0, period);
37
38 // 執行字串匹配
39 vector<int> kmp(const string& text, const string& pattern) {
40     vector<int> res;
41     if (pattern.empty()) return res;
42
43     vector<int> pi = build_pi(pattern);
44     for (int i = 0, j = -1; i < text.length(); ++i) {
45         while (j >= 0 && text[i] != pattern[j+1]) j = pi[j];
46         if (text[i] == pattern[j+1]) j++;
47     }

```

```

46         if (j == pattern.length() - 1) { // 完全匹配
47             res.push_back(i - j); // 紀錄起始 index
48             j = pi[j]; // 繼續尋找下一個可能的匹配
49         }
50     }
51     return res; // 回傳的這個 vector 包含所有完全匹配的 index 位置
52 }

```

4.2 Z-Algorithm 字串匹配

用法: $z[i]$ 代表 S 與 $S[i \dots N - 1]$ 的最長共同前綴。必須是 0-based index。

時間複雜度: $O(N)$

```

1 // z[i] 代表 s[0..n-1] 與 s[i..n-1] 的最長共同前綴 (LCP) 長度
2 // 若要找 pattern 在 text 中的位置, 可傳入 s = pattern + "$" + text
3 vector<int> z_function(const string& s) {
4     int n = s.length();
5     vector<int> z(n, 0);
6     // l, r 記錄目前「最靠右」的匹配區間 [l, r]
7     for (int i = 1, l = 0, r = 0; i < n; ++i) {
8         // 如果 i 在區間內, 可以直接利用對稱性 (i - l) 抄答案
9         if (i <= r) z[i] = min(r - i + 1, z[i - l]);
10
11         // 暴力往後擴展 (因為有前面的加速, 這裡實際上只會做很少次)
12         while (i + z[i] < n && s[z[i]] == s[i + z[i]]) {
13             z[i]++;
14         }
15
16         // 如果新匹配的區間超過了原有的 r, 更新 [l, r] 視窗
17         if (i + z[i] - 1 > r) {
18             l = i;
19             r = i + z[i] - 1;
20         }
21     }
22     return z;
23 }
24
25 string pattern = "aba";
26 string text = "abacaba";
27
28 string s = pattern + "$" + text;
29 vector<int> z = z_function(s);
30
31 for (int i = 0; i < s.size(); ++i) {
32     if (z[i] == pattern.size()) {
33         cout << "found at " << i - pattern.size() - 1 << "\n";
34     }
35 }
36 /*
37 found at 0
38 found at 4
39 */
40 }

```

4.3 Manacher 迴文判斷

用法: 求最長迴文字串。預處理字串塞入 # 將所有迴文轉為奇數長度。

時間複雜度: $O(N)$

```

1 // 求最長迴文
2 struct Manacher {
3     vector<int> p; // p[i] 記錄以 i 為中心的最長迴文半徑
4     string t; // 轉換後的字串
5
6     Manacher(const string& s) {
7         // 預處理: 將 "abc" 變成 "$a#b#c#" 來統一奇偶長度
8         // 頭尾放不同的字元 ($) 和 (^) 可省去邊界判斷
9         t = "$#";
10        for (char c : s) { t += c; t += '#'; }
11        t += '^';
12
13        int n = t.length();
14        p.assign(n, 0);
15        int c = 0, r = 0; // c: 目前最靠右的迴文中心, r: 該迴文的右界
16
17        for (int i = 1; i < n - 1; ++i) {
18            int mirror = 2 * c - i; // i 關於 c 的對稱點
19
20            // 如果 i 在迴文右界 r 內, 可以直接抄對稱點的答案
21            if (r > i) p[i] = min(r - i, p[mirror]);
22
23            // 暴力往外擴展半徑
24            while (t[i + 1 + p[i]] == t[i - 1 - p[i]]) {
25                p[i]++;
26            }
27
28            // 如果這個新迴文的右界超過了 r, 更新 c 和 r
29            if (i + p[i] > r) {
30                c = i;
31                r = i + p[i];
32            }
33        }
34
35        // 回傳原字串中的最長迴文長度
36        int get_max_len() {
37            int max_len = 0;
38            for (int len : p) max_len = max(max_len, len);
39            return max_len;
40        }
41
42        // 回傳所有出現的迴文, 包含位置不同的重複字串
43        // 如果要不重複就把 vector 改成 set 進行去重, 再 for 輸出就好
44        vector<string> get_all_palindromes(const string& s) {
45            vector<string> result;
46            // 遍歷所有可能的迴文中心
47            for (int i = 1; i < t.length() - 1; ++i) {
48                // p[i] 是以 i 為中心的最大迴文長度
49                // 每次往內縮 2 (砍掉頭尾字元), 即可得到以此為中心的所有較短迴文
50            }
51        }
52    }
53 }

```

```

51         for (int len = p[i]; len > 0; len -= 2) {
52             // 神奇的公式: (i - len) / 2 可以精準對應到原字串 s 的起點索引
53             int start_idx = (i - len) / 2;
54             result.push_back(s.substr(start_idx, len));
55         }
56     }
57     return result;
58 }
59
60 // 僅計算所有迴文字串的「總數量」
61 long long count_all_palindromes() {
62     long long total = 0;
63     for (int i = 1; i < t.length() - 1; ++i) {
64         // 對於中心 i, 包含的迴文數量恰好是 ceil(p[i] / 2)
65         total += (p[i] + 1) / 2;
66     }
67     return total;
68 }
69 };

```

4.4 Trie 字典樹

用法: 字串前綴統計或最大 XOR 問題。必須是 0-based index。

時間複雜度: $O(|S|)$

```

1 struct Trie {
2     // 如果是字元 size=26, 如果是二進制 XOR size=2
3     int tree[MAXN][size] = {};
4     int passed[MAXN] = {};
5     int stop[MAXN] = {};
6     // 以上陣列的都跳過 index 0
7     int cnt = 1;
8
9     void init() {
10        for (int i = 1; i <= cnt; ++i) {
11            for (int j = 0; j < size; ++j) tree[i][j] = 0;
12            passed[i] = 0;
13            stop[i] = 0;
14        }
15        cnt = 1;
16    }
17
18    void insert(string word) {
19        int curr = 1;
20        ++passed[curr]; // 經過點 ++passed
21        for (int i = 0; path; i < word.length(); ++i) {
22            path = word.at(i) - 'a'; // 求出一個位置的 index
23            if (tree[curr][path] == 0) tree[curr][path] = ++cnt; // 如果當前位置走到目標位置不存在, 就用 ++cnt 的值
24            curr = tree[curr][path]; // 繼續往下走
25            ++passed[curr]; // 記得經過就要 ++passed
26        }
27        ++stop[curr]; // 最後停住的地方要 ++stop
28    }
29
30    int search(string word) {
31        int curr = 1;
32        for (int i = 0; path; i < word.length(); ++i) {
33            path = word.at(i) - 'a';
34            if (tree[curr][path] == 0) return 0; // 如果從 curr 往 path 的路徑不存在, 代表搜尋的 word 不存在
35            curr = tree[curr][path]; // 否則繼續往下走
36        }
37        return stop[curr]; // 最後回傳在 curr 位置停下 (stop) 的次數
38    }
39
40    int prefixNumber(string word) {
41        int curr = 1;
42        for (int i = 0; path; i < word.length(); ++i) {
43            path = word.at(i) - 'a';
44            if (tree[curr][path] == 0) return 0;
45            curr = tree[curr][path];
46        }
47        return passed[curr]; // 和 search 方法一模一樣, 只是回傳的對象變成 passed
48    }
49
50    void erase(string word) {
51        if (search(word) == 0) return; // 如果找不到字就不管他
52        int curr = 1;
53        --passed[curr];
54        for (int i = 0; path; i < word.length(); ++i) {
55            path = word.at(i) - 'a';
56            if (--passed[tree[curr][path]] == 0) { // 如果 curr 往 path 的路徑上, p 值為 1, 代表後面的東西都不需要了
57                tree[curr][path] = 0; // 將 curr 不再連通 path, 直接捨棄後面所有的東西 (因為都會被減成 p=0)
58                return;
59            }
60            curr = tree[curr][path]; // 否則繼續往下走, 注意後面不需要再 --passed[curr], 因為在 if 區塊我們已經操作完了
61        }
62        --stop[curr]; // 最後 --stop[curr]
63    }
64 };

```

4.5 AC 自動機

用法: 多字串匹配 (Trie + KMP), 必須是 0-based index。fail[i] 指向當前狀態最長的、且在 Trie 中存在的後綴狀態。

時間複雜度: $O(N + M)$

```

1 struct AC_Automaton {
2     static const int MAXN = 1e5 + 10;
3     int tree[MAXN][size]; // size 是字元 26, 二進制 2
4     int fail[MAXN];
5     int stop[MAXN]; // 記錄有幾個字串在此結束
6     int cnt;
7
8     void init() {

```

```

9         cnt = 0;
10        new_node(); // root is 0
11    }
12
13    int new_node() {
14        fill(tree[cnt], tree[cnt] + 26, 0);
15        fail[cnt] = stop[cnt] = 0;
16        return cnt++;
17    }
18
19    // 1. 將所有關鍵字插入 Trie
20    void insert(const string& s) {
21        int curr = 0;
22        for (char c : s) {
23            int path = c - 'a';
24            if (!tree[curr][path]) {
25                tree[curr][path] = new_node();
26            }
27            curr = tree[curr][path];
28        }
29        stop[curr]++;
30    }
31
32    // 2. 建立 Fail 指標與字典圖優化 (使用 BFS)
33    void build() {
34        queue<int> q;
35        // 將 root (0) 的所有實際存在的第一層子節點推入 queue
36        for (int i = 0; i < 26; ++i) {
37            if (tree[0][i]) {
38                fail[tree[0][i]] = 0;
39                q.push(tree[0][i]);
40            }
41        }
42
43        while (!q.empty()) {
44            int u = q.front();
45            q.pop();
46
47            for (int i = 0; i < 26; ++i) {
48                if (tree[u][i]) {
49                    // 若子節點存在，它的 fail 指向父節點 fail 的對應子節點
50                    fail[tree[u][i]] = tree[fail[u]][i];
51                    q.push(tree[u][i]);
52                } else {
53                    // 優化：若子節點不存在，直接把這條路連向 fail 的對應節點
54                    // 這樣搜尋時就絕對不會遇到死胡同，也不用往回跳
55                    tree[u][i] = tree[fail[u]][i];
56                }
57            }
58        }
59    }
60
61    // 3. 搜尋文章，回傳所有關鍵字出現的總次數
62    int query(const string& text) {
63        int curr = 0;
64        int res = 0;
65        for (char c : text) {
66            curr = tree[curr][c - 'a']; // 感謝字典圖優化，無腦往下走即可
67        }
68        // 順著 fail 指標往上收集沿途所有匹配成功的字串 (例如配對到 "she"，
69        // 同時也配對到 "he")
70        int temp = curr;
71        while (temp != 0 && stop[temp] != -1) {
72            res += stop[temp];
73            stop[temp] = -1; // -1 是優化：已經算過的關鍵字不要重複算
74            temp = fail[temp];
75        }
76        return res;
77    }
78 };

```

5 圖論

需注意：
是否為全連通圖 (connected，只有一個 component)，對於連通圖要找 region 可以用 Euler's theorem $V-E+R=2$

undirected 無自環

directed

weighted 有權重 (點或邊)

multigraph 兩點之間有多條直接的路徑

有向圖的 outdegree, indegree，無向圖的 degree

表示圖的方式: Adjacency matrix, adjacency list

5.1 BFS 與拓樸排序

用法: 無權圖最短路徑 / 拓樸排序計算 In-degree 尋找相關順序，可判斷是否有環。

時間複雜度: $\mathcal{O}(V + E)$

```

1 struct BFS_Topo {
2     int n;
3     vector<vector<int>> adj; // adjacency list, 避免爆空間
4     vector<int> in_degree; // 紀錄入度，用於拓樸排序
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         in_degree.assign(n + 1, 0);
10    }
11
12    // 有向圖，u 指向 v (u 必須在 v 之前完成)
13    void add_edge(int u, int v) {

```

```

14        adj[u].push_back(v);
15        in_degree[v]++; // v 被 u 指到，入度 +1
16    }
17
18    // 1. 拓樸排序 (Kahn's Algorithm)
19    // 回傳拓樸排序的結果。若回傳的 vector 大小小於 n，代表圖中有環
20    vector<int> topo_sort() {
21        queue<int> q;
22        vector<int> res;
23
24        // 步驟一：將所有入度為 0 (沒有前置條件) 的點加入 queue
25        for (int i = 1; i <= n; ++i) {
26            if (in_degree[i] == 0) q.push(i);
27        }
28
29        // 步驟二：開始 BFS
30        while (!q.empty()) {
31            int u = q.front();
32            q.pop();
33            res.push_back(u);
34
35            // 拔掉 u 連出去的所有邊
36            for (int v : adj[u]) {
37                in_degree[v]--; // v 的前置條件少了一個
38                if (in_degree[v] == 0) { // 如果前置條件都滿足了，就推入 queue
39                    q.push(v);
40                }
41            }
42        }
43        return res; // 若 res.size() < n，說明有環，無法完成拓樸排序
44    }
45
46    // 2. 基礎 BFS：無權圖的單源最短路徑
47    vector<int> get_dist(int start) {
48        vector<int> dist(n + 1, -1);
49        queue<int> q;
50
51        dist[start] = 0;
52        q.push(start);
53
54        while (!q.empty()) {
55            int u = q.front();
56            q.pop();
57            for (int v : adj[u]) {
58                if (dist[v] == -1) { // 沒走過
59                    dist[v] = dist[u] + 1;
60                    q.push(v);
61                }
62            }
63        }
64        return dist;
65    }
66 };

```

5.2 DFS

用法: 用遞迴走訪。

時間複雜度: $\mathcal{O}(V + E)$

```

1 struct DFS {
2     int n;
3     vector<vector<int>> adj;
4     vector<bool> vis;
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         vis.assign(n + 1, false);
10        subtree_size.assign(n + 1, 0);
11    }
12
13    void add_edge(int u, int v) {
14        adj[u].push_back(v);
15        adj[v].push_back(u); // 無向圖就兩個都加
16    }
17
18    // 基礎 DFS：走訪與找連通塊
19    void dfs(int u) {
20        vis[u] = true;
21        // 如有需要，處理抵達 u 時的邏輯 (Pre/In/Post-order)
22
23        for (int v : adj[u]) {
24            if (!vis[v]) {
25                dfs(v);
26            }
27        }
28    }
29 };
30
31 /*
32 補充 1:
33 當題目節點數量 N <= 10，需要找一條 "最短路徑"，且每個邊的最大 degree 只有 2
34 可以使用 DFS 進行枚舉，從各個節點開始跑，對所有可能的路徑跑 DFS
35 邊跑邊更新最長路徑答案，跑完是  $\mathcal{O}(N!)$  ->  $N=10$  -> 小於 400 萬，能跑
36
37 補充 2:
38 如果發現題目要找樹的直徑時，可以使用兩次 dfs 求解 (前提是樹邊權不能是負的)
39 如：樹上最遠兩點距離？從一個人開始通知，每秒傳到相鄰節點，最慢多久可以全收到？路
40 徑唯一最長鏈？
41
42 vector<int> adj[1000];
43 int max_dist;
44 int farthest_node;
45
46 void dfs(int cur, int parent, int dist) {
47     if (dist > max_dist) {
48         max_dist = dist;
49         farthest_node = cur;
50     }
51     for (int &next : adj[cur]) {
52         if (next != parent) dfs(next, cur, dist+1);
53     }
54 }

```

```

52     }
53 }
54
55 void solve(){
56     int n, e;
57     cin >> n >> e;
58     for(int i=0; i<e; ++i){ // 讀入圖
59         int a, b;
60         cin >> a >> b;
61         adj[a].push_back(b);
62         adj[b].push_back(a);
63     }
64     max_dist = -1;
65     dist(1, -1, 0); // 一開始從任意點第一次 DFS
66
67     int first_farthest = farthest_node;
68     max_dist = -1;
69     dist(first_farthest, -1, 0);
70     // 最長距離為 max_dist, 點: first_farthest, farthest_node
71 }

```

5.3 Dijkstra 單源最短路徑

用法: 找單個起點的最短路徑, 不支援負權邊, 支援自環, 支援有/無向圖。

時間複雜度: $\mathcal{O}((V + E) \log V)$

```

1 struct Dijkstra {
2     // 定義 pii 為 pair< 距離, 節點 >, 預設由小到大排序 (pq 用)
3     using pii = pair<long long, int>;
4     const int MAXN = 2e5 + 10;
5     const long long INF = 1e18; // 使用 1e18 避免相加時 long long 溢位
6
7     vector<pair<int, long long>> adj[MAXN]; // adj[u] = {v, weight}
8     long long dist[MAXN]; // dist[i] 為起點到 i 的距離
9     int n;
10
11     // 初始化: 傳入節點數, 清空圖
12     void init(int _n) {
13         n = _n;
14         for (int i = 0; i <= n; ++i) {
15             adj[i].clear();
16         }
17     }
18
19     void add_edge(int u, int v, long long w) {
20         adj[u].push_back({v, w});
21         // 若為無向圖, 須加上 adj[v].push_back({u, w});
22     }
23
24     void solve(int start) {
25         fill(dist, dist + n + 1, INF); // 將距離陣列初始化為無限大
26         // pq 存起點到某一點的距離, 用距離的 Min-Heap
27         priority_queue<pii, vector<pii>, greater<pii>> pq;
28
29         dist[start] = 0;
30         pq.push({0, start});
31
32         while (!pq.empty()) {
33             long long d = pq.top().first;
34             int u = pq.top().second;
35             pq.pop();
36
37             // 懶惰刪除 (Lazy Deletion): 如果取出時發現已經有更好的距離, 直接丟棄
38             if (d > dist[u]) continue;
39
40             for (auto& edge : adj[u]) {
41                 int v = edge.first;
42                 long long w = edge.second;
43
44                 // 鬆弛 (Relax)
45                 if (dist[u] + w < dist[v]) {
46                     dist[v] = dist[u] + w;
47                     pq.push({dist[v], v});
48                 }
49             }
50         }
51     }
52 };

```

5.4 SPFA 單源最短路徑

用法: 也是找單個起點的最短路徑, 但支援負權邊。只在“可能有負邊”或“必須檢查負環”時才用

時間複雜度: 預期 $\mathcal{O}(V + E)$, 最差 (網格圖) $\mathcal{O}(VE)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 1005;
5 const long long INF = 1e18;
6
7 struct Edge {
8     int v;
9     long long w;
10 };
11
12 vector<Edge> G[MAXN]; // adjacency list 存圖
13 long long dist[MAXN]; // 起點到任意點的距離
14 int n, m; // n 個點, m 個邊
15
16 bool SPFA(int start) { // 回傳 false 代表圖中存在負環; 回傳 true 代表最短路徑計算成功
17     bool inq[MAXN]; // inq[i] 記錄點 i 目前是否在 queue 裡面
18     int cnt[MAXN]; // cnt[i] 記錄點 i 進入 queue 的總次數
19     queue<int> q;
20
21     fill(dist, dist + n + 1, INF);

```

```

22     memset(inq, false, sizeof(inq));
23     memset(cnt, 0, sizeof(cnt));
24
25     dist[start] = 0; // 從 start 節點開始
26     q.push(start);
27     inq[start] = true;
28     cnt[start] = 1;
29
30     while (!q.empty()) {
31         int u = q.front();
32         q.pop();
33         inq[u] = false; // 拿出來後就不在 queue 裡了
34
35         for (auto& edge : G[u]) {
36             int v = edge.v;
37             long long w = edge.w;
38
39             // 鬆弛操作 (relax)
40             if (dist[u] + w < dist[v]) {
41                 dist[v] = dist[u] + w;
42
43                 // 如果被鬆弛的點不在 queue 裡面, 就把它加進去
44                 if (!inq[v]) {
45                     q.push(v);
46                     inq[v] = true;
47                     cnt[v]++;
48
49                     if (cnt[v] >= n) return false; // 【關鍵】一個點入隊次數 >= n (點總數), 必定有負環
50                 }
51             }
52         }
53     }
54     return true;
55 }

```

5.5 Floyd-Warshall 多源最短路徑

用法: 求任意兩點之間距離。利用滾動陣列降維, **k** 必須在最外層迴圈。注意避開 INF 加上負權邊的陷阱。

時間複雜度: 時間 $\mathcal{O}(V^3)$, 空間 $\mathcal{O}(V^2)$

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const int MAXN = 505;
5 const long long INF = 1e18;
6 long long dist[MAXN][MAXN];
7 int n, m;
8
9 void init() {
10     for (int i = 1; i <= n; ++i) {
11         for (int j = 1; j <= n; ++j) {
12             dist[i][j] = (i == j) ? 0 : INF;
13         }
14     }
15 }
16
17 // 注意: 讀邊時若為多重邊 (Multigraph, 即 A->B 不只一條 direct path), 需寫成
18 // dist[u][v] = min(dist[u][v], w);
19
20 void floyd() {
21     // 中繼點 k 必須在最外層!
22     for (int k = 1; k <= n; ++k) {
23         for (int i = 1; i <= n; ++i) {
24             for (int j = 1; j <= n; ++j) {
25                 // 必須確認 i->k 和 k->j 都是通的
26                 if (dist[i][k] < INF && dist[k][j] < INF) {
27                     dist[i][j] = min(dist[i][j], dist[i][k] + dist[k][j]);
28                 }
29             }
30         }
31     }
32 }

```

5.6 二分圖判定 (著色問題)

用法: 規定相鄰節點必須塗上不同顏色, 如果在著色過程中, 發現相鄰節點已經被塗上相同的顏色, 則代表此圖包含奇數環 Odd Cycle, 不是二分圖。已考量到不連通圖的情況

時間複雜度: $\mathcal{O}(V + E)$

```

1 struct BipartiteCheck {
2     int n;
3     vector<vector<int>> adj;
4     vector<int> color; // 0: 未著色, 1: 顏色 A, -1: 顏色 B
5
6     void init(int _n) {
7         n = _n;
8         adj.assign(n + 1, vector<int>());
9         color.assign(n + 1, 0); // 初始化全為未著色
10    }
11
12    void add_edge(int u, int v) {
13        adj[u].push_back(v);
14        adj[v].push_back(u); // 二分圖判斷通常針對無向圖
15    }
16
17    bool solve() {
18        bool is_bipartite = true;
19
20        // 圖可能有多個連通分量 (Components), 必須確保每個節點都被檢查到
21        for (int i = 1; i <= n; ++i) {
22            if (color[i] == 0) { // 遇到未著色的節點, 當作新 Component 的起點
23                queue<int> q;
24                q.push(i);
25                color[i] = 1; // 塗上第一種顏色
26
27                while (!q.empty()) {

```

```

28         int u = q.front();
29         q.pop();
30
31         for (int v : adj[u]) {
32             if (color[v] == 0) {
33                 // 相鄰節點未著色，塗上相反的颜色 (-color[u]) 並加入 queue
34                 color[v] = -color[u];
35                 q.push(v);
36             } else if (color[v] == color[u]) {
37                 // 相鄰節點已著色，且與自己同色 -> 產生奇數環，非二分圖
38                 is_bipartite = false;
39                 return false; // 提早結束，不需繼續檢查
40             }
41         }
42     }
43 }
44 }
45 return is_bipartite;
46 }
47 };

```

5.7 割點與雙連通分量

用法：處理網路穩定性、單點故障分析。透過 DFS 一次遍歷找出所有的割點以及 v-BCC。由於一個割點可能同時屬於多個 BCC，我們利用 stack 記錄 DFS 走過的點，當發現割點條件成立時，再把點從 stack 彈出形成 BCC。

時間複雜度： $\mathcal{O}(V + E)$

```

1  /*
2  dfn[u]：DFS 時，節點 u 被拜訪到的「時間戳記」或「順序」。一旦給定就不會再改變。
3
4  low[u]：從節點 u 的子樹出發，在不經過 u 的那條父節點邊 的前提下，透過 back-edge 能到達的所有節點中，最小的 dfn 值。
5  */
6
7  /*
8  割點 (Articulation Point)：
9  定義：在無向連通圖中，如果移除某個節點及其所有相連的邊後，圖被切斷成兩個或多個不
    相連的部分，該點就是割點。
10 用途：尋找網路、電路或交通系統中的「單點故障」瓶頸。只要這個點壞了，整個系統就會癱
    瘓。
11 判斷條件：
12 若 u 是 DFS 的根節點 (Root)，且它有 2 個以上的子樹，則 u 是割點。
13 若 u 不是根節點，且存在至少一個子節點 v 滿足 low[v] >= dfn[u]。這代表 v 及其子孫
    「最多只能爬回 u 自己」，無法繞過 u 抵達更上層的祖先。如果把 u 拔掉，v 就會跟上面斷
    聯，因此 u 是割點。
14
15 點雙連通分量 (Vertex Biconnected Component)：
16 定義：一個不包含任何割點的「極大連通子圖」。在 v-BCC 中，任意兩點之間至少有兩條點
    不重複的路徑。
17 用途：用來縮點。把一個個 v-BCC 視為一個大節點，可以將複雜的圖形轉化為「樹」的結
    構，進而利用 LCA 或樹上 DP 來處理複雜的圖上路徑詢問。
18  */
19
20 struct BCC {
21     int n, timer;
22     vector<vector<int>> adj;
23     vector<int> dfn, low;
24     vector<bool> is_cut; // 紀錄是否為割點
25     vector<vector<int>> bcc_nodes; // 儲存每個 BCC 內的點集
26     stack<int> st;
27
28     void init(int _n) {
29         n = _n;
30         timer = 0;
31         adj.assign(n + 1, vector<int>());
32         dfn.assign(n + 1, 0);
33         low.assign(n + 1, 0);
34         is_cut.assign(n + 1, false);
35         bcc_nodes.clear();
36         while (!st.empty()) st.pop();
37     }
38
39     void add_edge(int u, int v) {
40         adj[u].push_back(v);
41         adj[v].push_back(u);
42     }
43
44     void dfs(int u, int p = -1) {
45         dfn[u] = low[u] = ++timer;
46         st.push(u);
47         int children = 0; // 紀錄 DFS 樹上的子節點數量
48
49         for (int v : adj[u]) {
50             if (v == p) continue; // 不走回頭路 (父節點)
51
52             if (dfn[v]) {
53                 // v 已經拜訪過，代表這是一條 Back-edge (回邊)
54                 low[u] = min(low[u], dfn[v]);
55             } else {
56                 // v 沒拜訪過，代表這是一條 Tree-edge (樹邊)
57                 children++;
58                 dfs(v, u);
59
60                 // 子節點回來後，用子節點的 low 更新自己的 low
61                 low[u] = min(low[u], low[v]);
62
63                 // 判斷割點與收集 BCC 的核心條件
64                 if (low[v] >= dfn[u]) {
65                     is_cut[u] = true;
66                     vector<int> comp;
67
68                     // 從 stack 把屬於這個 BCC 的點全部彈出
69                     while (true) {
70                         int w = st.top();

```

```

71         st.pop();
72         comp.push_back(w);
73         if (w == v) break; // 注意：退到 v 就停下
74     }
75     // 割點 u 會同時屬於多個 BCC，把它加進去但【不要】從 stack
    彈出它
76     comp.push_back(u);
77     bcc_nodes.push_back(comp);
78 }
79 }
80 }
81
82 // 根節點特判：DFS 起點如果有小於 2 個子樹，則不是割點
83 if (p == -1 && children < 2) {
84     is_cut[u] = false;
85 }
86 }
87
88 void solve() {
89     for (int i = 1; i <= n; ++i) {
90         if (!dfn[i]) {
91             dfs(i);
92
93             // 處理極端情況：圖中若存在孤立的單個點，將其視為一個 BCC
94             if (bcc_nodes.empty() && n == 1) {
95                 bcc_nodes.push_back({1});
96             }
97         }
98     }
99 }
100 };

```

5.8 匈牙利演算法

用法：無權二分圖最大匹配，透過回溯尋找增廣路徑，以達到最大匹配。分為左 (S)、右 (T) 兩側，以及 left 陣列紀錄右側所選的對象

時間複雜度： $\mathcal{O}(V \cdot E)$

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  const int MAXN = 30005; // 依題目需求調整右側/左側最大點數
5  int n, m, e; // n: 左側點數, m: 右側點數
6  vector<int> G[MAXN]; // G[u] 存左側點 u 連向右側的哪些點
7  int L[MAXN]; // L[v] 記錄右側點 v 配對到的左側點
8  bool T[MAXN]; // T[v] 記錄右側點 v 在單次 DFS 中是否已拜訪
9
10 bool match(int u) { // 核心 DFS：幫左側點 u 尋找增廣路徑
11     for (int v : G[u]) {
12         if (T[v]) continue;
13         T[v] = true;
14
15         // 若右側點 v 還沒選對象，或其原本對象可以讓出位子
16         if (L[v] == -1 || match(L[v])) {
17             L[v] = u; // 配對成功
18             return true;
19         }
20     }
21     return false;
22 }
23
24 int solve() { // 執行最大匹配
25     int ans = 0;
26     memset(L, -1, sizeof(L)); // 依照需求假設點編號為 0-based 或 1-based
27
28     // 假設左側點編號為 1 ~ n (若為 0 ~ n-1 請自行改迴圈範圍)
29     for (int i = 1; i <= n; ++i) {
30         memset(T, false, sizeof(T));
31         if (match(i)) {
32             ans++;
33         }
34     }
35     return ans;
36 }
37
38 int main() {
39     while (cin >> n >> m) {
40         for (int i = 0; i <= n; ++i) {
41             G[i].clear();
42         }
43
44         int u, v; // 讀取每個邊，存到 adjacency list 中
45         for (int i = 0; i < e; ++i) {
46             cin >> u >> v;
47             G[u].push_back(v);
48         }
49
50         // 依照需求做更改，這裡的範例是：找到的最大匹配數等於邊的數量，即完成最大匹配
51         if (solve() == e) cout << "YES\n";
52         else cout << "NO\n";
53     }
54     return 0;
55 }

```

5.9 KM 演算法

用法：適用於左右節點數皆為 N 的帶權完全二分圖

時間複雜度： $\mathcal{O}(N^3)$

```

1  /*
2  前提：完美匹配與虛擬節點
3  KM 演算法要求左右兩側的節點數必須相等 (形成完全二分圖)
4  如果在題目中左右節點數不同，或者某些點之間沒有連線，我們必須加入「虛擬節點」與「權
    重為 0 (或極小值) 的虛擬邊」來補齊，才能讓 KM 演算法順利運作。
5  */
6
7  struct KM {
8      int n;

```

```

9     vector<vector<long long>> weight; // weight[u][v] 表示左 u 到右 v 的權重
10    vector<long long> lx, ly, slack; // lx, ly 為左右點的「期望值 (頂標)」,
    slack 為鬆弛量
11    vector<int> match_y; // match_y[v] = u 代表右側 v 匹配給左側
    u
12    vector<int> pre; // 紀錄右側節點的交替樹路徑, 用於 BFS 回
    溯
13    vector<bool> vis_y; // 紀錄右側節點是否在當前交替樹中
14    const long long INF = 1e18;
15
16    void init(int _n) {
17        n = _n;
18        // 為了方便, 採用 1-based index, 0 作為虛擬的空節點
19        weight.assign(n + 1, vector<long long>(n + 1, 0));
20        lx.assign(n + 1, 0);
21        ly.assign(n + 1, 0);
22        match_y.assign(n + 1, 0);
23    }
24
25    void add_edge(int u, int v, long long w) {
26        weight[u][v] = max(weight[u][v], w); // 若有多重邊, 保留權重最大的
27    }
28
29    // BFS 尋找增廣路徑並更新頂標
30    void bfs(int root) {
31        slack.assign(n + 1, INF);
32        vis_y.assign(n + 1, false);
33        pre.assign(n + 1, 0);
34
35        int y = 0;
36        match_y[0] = root; // 將起始點放在虛擬節點 match_y[0]
37
38        while (true) {
39            int x = match_y[y]; // 當前考慮的左側節點
40            long long delta = INF;
41            int next_y = 0;
42            vis_y[y] = true;
43
44            // 遍歷所有右側節點, 計算 slack 量
45            for (int i = 1; i <= n; ++i) {
46                if (!vis_y[i]) {
47                    long long diff = lx[x] + ly[i] - weight[x][i];
48                    if (diff < slack[i]) {
49                        slack[i] = diff;
50                        pre[i] = y; // 紀錄從哪個右側節點轉移過來
51                    }
52                    if (slack[i] < delta) {
53                        delta = slack[i];
54                        next_y = i;
55                    }
56                }
57            }
58
59            // 更新交替樹中所有點的頂標 (期望值)
60            for (int i = 0; i <= n; ++i) {
61                if (vis_y[i]) {
62                    lx[match_y[i]] -= delta;
63                    ly[i] += delta;
64                } else {
65                    slack[i] -= delta;
66                }
67            }
68
69            y = next_y; // 走到下一個右側節點
70            if (match_y[y] == 0) break; // 找到未匹配的右側節點, 增廣路徑結束
71        }
72
73        // 回溯並更新匹配狀態
74        while (y != 0) {
75            match_y[ly] = match_y[pre[y]];
76            y = pre[y];
77        }
78    }
79
80    long long solve() {
81        // 初始化左側頂標為連接邊的最大權重
82        for (int i = 1; i <= n; ++i) {
83            lx[i] = -INF;
84            for (int j = 1; j <= n; ++j) {
85                lx[i] = max(lx[i], weight[i][j]);
86            }
87        }
88
89        // 對每一個左側節點進行匹配
90        for (int i = 1; i <= n; ++i) {
91            bfs(i);
92        }
93
94        // 計算最大權重總和
95        long long max_weight = 0;
96        for (int i = 1; i <= n; ++i) {
97            if (match_y[i] != 0) {
98                max_weight += weight[match_y[i]][i];
99            }
100        }
101        return max_weight;
102    }
103 };

```

5.10 Dinic 最大流

用法: 建圖時需加入反向邊做為退流機制。多筆測資要記得清空 adj。
時間複雜度: $\mathcal{O}(V^2 E)$, 二分圖匹配 $\mathcal{O}(E\sqrt{V})$

```

1 #include <vector>
2 #include <queue>
3 #include <algorithm>
4 using namespace std;
5
6 const int MAXV = 1005; // 依據題目點數調整

```

```

7 const long long INF = 1e18;
8
9 // 儲存邊的結構
10 struct Edge {
11     int to;
12     long long cap, flow;
13     int rev; // 紀錄這條邊的「反向邊」在對方 adj 陣列裡的位置 (index)
14 };
15
16 vector<Edge> adj[MAXV];
17 int level[MAXV]; // BFS 分層圖, 紀錄起點到該點的距離
18 int ptr[MAXV]; // 【當前弧優化】紀錄 DFS 目前探索到哪條邊, 避免走廢邊
19 int n, s, t; // 總點數, 源點 (Source), 匯點 (Sink)
20
21 // 加入一條有向邊 (若是無向邊, 請再呼叫一次反方向, 或直接把反向邊的 cap 也設為
    w)
22 void add_edge(int from, int to, long long w) {
23     adj[from].push_back({to, w, 0, (int)adj[to].size()});
24     adj[to].push_back({from, 0, 0, (int)adj[from].size() - 1});
25 }
26
27 // 1. BFS 建立分層圖
28 bool bfs() {
29     fill(level, level + n + 1, -1);
30     level[s] = 0;
31     queue<int> q;
32     q.push(s);
33
34     while (!q.empty()) {
35         int v = q.front();
36         q.pop();
37         for (auto& edge : adj[v]) {
38             // 如果這條邊還有剩餘容量, 且終點還沒被分層
39             if (edge.cap - edge.flow > 0 && level[edge.to] == -1) {
40                 level[edge.to] = level[v] + 1;
41                 q.push(edge.to);
42             }
43         }
44     }
45     return level[t] != -1; // 回傳是否還能走到匯點 T
46 }
47
48 // 2. DFS 尋找增廣路徑並推播流量
49 long long dfs(int v, long long pushed) {
50     if (pushed == 0) return 0;
51     if (v == t) return pushed;
52
53     // 利用 ptr[v] 紀錄上次走到哪一條邊, 這是 Dinic 不會 TLE 的關鍵!
54     // 注意 cid 必須宣告為 reference (&) 才能真正更新 ptr 陣列
55     for (int& cid = ptr[v]; cid < adj[v].size(); ++cid) {
56         auto& edge = adj[v][cid];
57         int tr = edge.to;
58
59         // 條件: 只能往下一層走, 且邊要有剩餘容量
60         if (level[v] + 1 != level[tr] || edge.cap - edge.flow == 0) continue;
61
62         long long push = dfs(tr, min(pushed, edge.cap - edge.flow));
63         if (push == 0) continue;
64
65         // 更新正向與反向邊的流量
66         edge.flow += push;
67         adj[tr][edge.rev].flow -= push;
68         return push;
69     }
70     return 0;
71 }
72
73 // 主函式: 計算最大流
74 long long dinic(int _n, int _s, int _t) {
75     n = _n; s = _s; t = _t;
76     long long flow = 0;
77
78     // 只要還能走到匯點, 就持續建分層圖
79     while (bfs()) {
80         fill(ptr, ptr + n + 1, 0); // 每次重新分層後, DFS 指標要歸零
81
82         // 在同一層圖中, 盡可能把所有增廣路徑榨乾
83         while (long long pushed = dfs(s, INF)) {
84             flow += pushed;
85         }
86     }
87     return flow;
88 }
89
90 // 若有多筆測資, 請記得在 main 裡面先跑:
91 // for(int i = 0; i <= n; ++i) adj[i].clear();

```

6 幾何

A. Circle and Line

點 $P(x_0, y_0)$
到直線 $L: ax + by + c = 0$ 的距離

$$d(P, L) = \frac{|ax_0 + by_0 + c|}{\sqrt{a^2 + b^2}}$$

兩平行直線 $L_1: ax + by + c_1 = 0$
與 $L_2: ax + by + c_2 = 0$ 的距離

$$d(L_1, L_2) = \frac{|c_1 - c_2|}{\sqrt{a^2 + b^2}}$$

B. Triangle

設三角形頂點為 $A(x_1, y_1), B(x_2, y_2), C(x_3, y_3)$
 點 A, B, C 的對邊長分別為 a, b, c
 三角形面積為 Δ
 重心為 (G_x, G_y) ，內心為 (I_x, I_y) ，
 外心為 (O_x, O_y) 和垂心為 (H_x, H_y)

$$\Delta = \frac{1}{2} \begin{vmatrix} x_1 & y_1 & 1 \\ x_2 & y_2 & 1 \\ x_3 & y_3 & 1 \end{vmatrix}$$

$$G_x = \frac{1}{3}(x_1 + x_2 + x_3)$$

$$G_y = \frac{1}{3}(y_1 + y_2 + y_3)$$

$$I_x = \frac{ax_1 + bx_2 + cx_3}{a + b + c}$$

$$I_y = \frac{ay_1 + by_2 + cy_3}{a + b + c}$$

$$O_x = \frac{1}{4\Delta} \begin{vmatrix} x_1^2 + y_1^2 & y_1 & 1 \\ x_2^2 + y_2^2 & y_2 & 1 \\ x_3^2 + y_3^2 & y_3 & 1 \end{vmatrix}$$

$$O_y = \frac{1}{4\Delta} \begin{vmatrix} x_1 & x_1^2 + y_1^2 & 1 \\ x_2 & x_2^2 + y_2^2 & 1 \\ x_3 & x_3^2 + y_3^2 & 1 \end{vmatrix}$$

$$H_x = -\frac{1}{2\Delta} \begin{vmatrix} x_2x_3 + y_2y_3 & y_1 & 1 \\ x_1x_3 + y_1y_3 & y_2 & 1 \\ x_1x_2 + y_1y_2 & y_3 & 1 \end{vmatrix}$$

$$H_y = -\frac{1}{2\Delta} \begin{vmatrix} x_1 & x_2x_3 + y_2y_3 & 1 \\ x_2 & x_1x_3 + y_1y_3 & 1 \\ x_3 & x_1x_2 + y_1y_2 & 1 \end{vmatrix}$$

任意三角形，重心、外心、垂心共線

$$G_x = \frac{2}{3}O_x + \frac{1}{3}H_x$$

$$G_y = \frac{2}{3}O_y + \frac{1}{3}H_y$$

C. Quadrilateral

任意凸四邊形 $ABCD$ 的四邊長分別為 a, b, c, d
 且已知 $\angle A + \angle C$ ，則四邊形 $ABCD$ 的面積為

$$\sqrt{(s-a)(s-b)(s-c)(s-d) - \Delta}$$

where

$$s = \frac{a + b + c + d}{2}$$

$$\Delta = abcd \cos^2 \left(\frac{A + C}{2} \right)$$

特例：若 $ABCD$ 為圓內接四邊形，則 $\Delta = 0$

若只知道其中一角，則可用餘弦定理

$$c^2 = a^2 + b^2 - 2ab \cos(\angle C)$$

求出對角線長，再用海龍計算兩個三角形面積即可。

6.1 點與向量基礎 (Point & Vector)

用法：幾何基礎，定義 struct 與 operator。包含浮點數誤差處理 sign，以及核心工具 dot (內積) 與 cross (外積)。

時間複雜度: $\mathcal{O}(1)$

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 const double EPS = 1e-9; // 用來處理浮點數誤差的極小值
5
6 // 符號判斷函數 (極度重要!)
7 // 判斷一個浮點數是 正數 (1)、負數 (-1) 還是 0(0)
8 int sign(double x) {
9     if (fabs(x) < EPS) return 0;
10    return x < 0 ? -1 : 1;
11 }
12
13 struct Point {
14     double x, y; // 通用 double
15     Point(double _x = 0, double _y = 0) : x(_x), y(_y) {}
16
17     // 向量加法與常數乘除
```

```
18     Point operator+(const Point& p) const { return Point(x + p.x, y + p.y); }
19     Point operator-(const Point& p) const { return Point(x - p.x, y - p.y); }
20     Point operator*(double d) const { return Point(x * d, y * d); }
21     Point operator/(double d) const { return Point(x / d, y / d); }
22
23     // 判斷兩點是否相等 (利用 sign 來避免浮點數誤差)
24     bool operator==(const Point& p) const {
25         return sign(x - p.x) == 0 && sign(y - p.y) == 0;
26     }
27
28     bool operator<(const Point& p) const {
29         if (x != p.x) return x < p.x;
30         return y < p.y;
31     }
32 };
33
34 using Vector = Point; // 在幾何中，向量和點的表示法一樣，給它一個別名增加可讀性
35
36 // 1. 內積 (Dot Product)，計算方式：x1*x2 + y1*y2
37 // 實戰意義：用來判斷兩向量夾角。大於 0 為銳角，小於 0 為鈍角，等於 0 為垂直。
38 double dot(Vector a, Vector b) {
39     return a.x * b.x + a.y * b.y;
40 }
41
42 // 2. 外積 (Cross Product)，計算方式：x1*y2 - x2*y1
43 // 實戰意義：用來判斷方向與面積。
44 // 若 cross(a, b) > 0，代表 b 在 a 的逆時針方向 (偏左)
45 // 若 cross(a, b) < 0，代表 b 在 a 的順時針方向 (偏右)
46 // 若 cross(a, b) == 0，代表 a 和 b 平行或共線
47 double cross(Vector a, Vector b) {
48     return a.x * b.y - a.y * b.x;
49 }
```

6.2 判斷線段、矩形相交 (Segment Intersection)

用法：利用外積進行跨立實驗。避開斜率除以零的問題，可處理端點相切與共線重疊。

時間複雜度: $\mathcal{O}(1)$

```
1 #include <algorithm>
2 using namespace std;
3
4 // 依賴上一節的 Point 結構與 sign, cross 函數
5
6 // 輔助函數：判斷點 P 是否在線段 AB 的「邊界框 (Bounding Box)」內
7 // 實戰意義：當三點共線時，單靠外積無法確認 P 是否在線段上 (可能在延長線上)
8 bool on_segment(Point p, Point a, Point b) {
9     return p.x >= min(a.x, b.x) && p.x <= max(a.x, b.x) &&
10        p.y >= min(a.y, b.y) && p.y <= max(a.y, b.y);
11 }
12
13 // 核心函數：判斷線段 AB 與線段 CD 是否相交
14 bool segment_intersect(Point a, Point b, Point c, Point d) {
15     // 1. 計算四個外積值 (跨立實驗)
16     // 意義：判斷 C 點和 D 點分別在直線 AB 的哪一側
17     double c1 = cross(b - a, c - a);
18     double c2 = cross(b - a, d - a);
19
20     // 意義：判斷 A 點和 B 點分別在直線 CD 的哪一側
21     double c3 = cross(d - c, a - c);
22     double c4 = cross(d - c, b - c);
23
24     // 2. 規範相交 (Strictly Intersect)
25     // 如果 C, D 在 AB 兩側 (外積正負號相反，相乘 < 0)
26     // 且 A, B 在 CD 兩側，則保證兩線段相交
27     if (sign(c1) * sign(c2) < 0 && sign(c3) * sign(c4) < 0) {
28         return true;
29     }
30
31     // 3. 特殊情況處理 (端點相交 或 共線重疊)
32     // 如果外積為 0，代表該點在另一條直線上。
33     // 接著必須用 on_segment 確保它真的落在「線段區間」內，而不是在延長線上。
34     if (sign(c1) == 0 && on_segment(c, a, b)) return true;
35     if (sign(c2) == 0 && on_segment(d, a, b)) return true;
36     if (sign(c3) == 0 && on_segment(a, c, d)) return true;
37     if (sign(c4) == 0 && on_segment(b, c, d)) return true;
38
39     return false; // 以上皆非，則不相交
40 }
```

6.3 點與線段關係 (Point & Line)

用法：判斷 1. 點是否在線段上 2. 求點到直線/線段最短距離。利用內外積避免解方程式與浮點數除以零。

時間複雜度: $\mathcal{O}(1)$

```
1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point, Vector, sign, cross, dot
5
6 // 輔助函數：求向量長度
7 // 原理：向量自己跟自己做內積，等於長度的平方
8 double length(Vector a) {
9     return sqrt(dot(a, a));
10 }
11
12 // 1. 判斷點 P 是否在線段 AB 上
13 // 原理：
14 // (1) cross(p-a, b-a) == 0 代表三點共線
15 // (2) dot(a-p, b-p) <= 0 代表 PA 向量與 PB 向量方向相反 (即 P 夾在 A, B 中間)
16 bool on_segment(Point p, Point a, Point b) {
17     return sign(cross(p - a, b - a)) == 0 && sign(dot(a - p, b - p)) <= 0;
18 }
19
20 // 2. 點 P 到「直線」AB 的距離
21 // 原理：平行四邊形面積 = 底 * 高 => 高 = 面積 / 底
22 double dist_to_line(Point p, Point a, Point b) {
23     // 面積剛好是外積的絕對值，底邊長是 AB 的長度
```

```

24     return fabs(cross(b - a, p - a)) / length(b - a);
25 }
26
27 // 3. 點 P 到「線段」AB 的最短距離 (賽場極易錯考點)
28 // 原理：用內積判斷 P 點的「投影」是否掉出線段外
29 double dist_to_segment(Point p, Point a, Point b) {
30     if (a == b) return length(p - a); // A, B 根本是同一個點
31
32     // 鈍角測試 1：角 PAB 是鈍角，代表 P 在 A 的外側，離 A 最近
33     if (sign(dot(b - a, p - a)) < 0) return length(p - a);
34
35     // 鈍角測試 2：角 PBA 是鈍角，代表 P 在 B 的外側，離 B 最近
36     if (sign(dot(a - b, p - b)) < 0) return length(p - b);
37
38     // 如果都不是鈍角，代表 P 的投影乖乖落在線段 AB 上，距離就是點到直線距離
39     return dist_to_line(p, a, b);
40 }

```

6.4 任意多邊形面積

用法：利用外積求多邊形面積 (鞋帶公式)。頂點必須依順時針或逆時針順序給定。支援凹凸多邊形。

```

時間複雜度： $\mathcal{O}(N)$ 
1 #include <cmath>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXV = 1005;
7 Point poly[MAXV]; // 儲存多邊形的頂點 (必須依照順時針或逆時針順序排列)
8 int n; // 頂點數量
9
10 // 求任意多邊形面積 (Shoelace Formula)
11 double polygon_area() {
12     double area = 0;
13     for (int i = 0; i < n; ++i) {
14         // 讓最後一個點與第 0 個點連線，形成封閉圖形
15         int next = (i + 1) % n;
16         area += cross(poly[i], poly[next]);
17     }
18     // 外積和有可能是負的 (取決於順逆時針)，所以取絕對值並除以 2
19     return fabs(area) / 2.0;
20 }
21
22 /*
23 補充：假設兩個矩形的左下角與右上角座標分別為
24 矩形 A：左下 (x1, y1)，右上 (x2, y2)
25 矩形 B：左下 (x3, y3)，右上 (x4, y4)
26
27 把矩形投影到 X 軸上，就變成了兩個線段 [x1, x2] 與 [x3, x4]
28 它們重疊的長度，等於「兩個右邊界的較小值」減去「兩個左邊界的較大值」
29 如果算出來是負數，代表它們根本沒碰到，直接跟 0 取最大值即可
30
31 因此計算矩形交集面積即為
32 int W = max(0, min(x2, x4) - max(x1, x3));
33 int H = max(0, min(y2, y4) - max(y1, y3));
34 int intersect_area = W * H;
35 */

```

6.5 凸包 Convex Hull

用法：求包圍所有點的最小凸多邊形。先排 X 再排 Y 。由左至右建下半部，由右至左建上半部。利用外積判斷並踢除右轉點。

```

時間複雜度： $\mathcal{O}(N \log N)$ 
1 #include <algorithm>
2 using namespace std;
3
4 // 依賴前面的 Point 結構與 cross 函數
5
6 const int MAXN = 100005;
7 Point pts[MAXN]; // 存放所有輸入的點
8 Point hull[MAXN]; // 存放最終構成凸包的點
9 int n; // 輸入點的數量
10
11 // 回傳凸包上的頂點數量
12 int convex_hull() {
13     // 點太少直接就是凸包
14     if (n <= 2) {
15         for (int i = 0; i < n; ++i) hull[i] = pts[i];
16         return n;
17     }
18
19     // 1. 將所有點由左至右、下至上排序
20     sort(pts, pts + n);
21
22     int m = 0; // m 代表目前凸包 (hull) 裡面有幾個點
23
24     // 2. 蓋下半部凸包 (Lower Hull) - 從左走到右
25     for (int i = 0; i < n; ++i) {
26         // 【核心邏輯】
27         // cross <= 0 代表「往右轉」或是「直走」。
28         // 凸包的邊緣必須一直「往左轉」，如果發現往右轉 (凹進去了)，就把最後一個點
29         // 踢掉！
30         // (若想保留共線的點，把 <= 改成 < 即可)
31         while (m >= 2 && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
32             m--; // 踢掉最後一個點
33         }
34         hull[m++] = pts[i]; // 把新點加進來
35
36     }
37
38     // 3. 蓋上半部凸包 (Upper Hull) - 從右走回左
39     // 注意：下半部已經蓋了 m 個點，為了不踢掉下半部的點，我們設定底線為 t
40     int t = m + 1;
41     for (int i = n - 2; i >= 0; --i) {
42         // 邏輯跟上面一模一樣，只是倒著走

```

```

41         while (m >= t && cross(hull[m - 1] - hull[m - 2], pts[i] - hull[m - 2]) <= 0) {
42             m--;
43         }
44         hull[m++] = pts[i];
45     }
46
47     // 4. 起點 (最左下的點) 會在頭尾被重複加入一次，所以總數量要減 1
48     return m - 1;
49     // 如果需要知道所有選擇的點，就是 hull[0]-hull[m-1]
50     // 然後有了這些點又可以用面積公式直接得到整個的面積
51 }

```

7 樹論

7.1 Disjoint Set

用法：解決集合、快速合併的問題。優化必須路徑壓縮，可選小掛大時間複雜度： $\mathcal{O}(1)$

```

1 struct DSU {
2     vector<int> f, sz; // 避免使用 size 撞名
3
4     void init(n) {
5         f.resize(n);
6         sz.assign(n, 1); // 將每個獨立集合的大小初始化為 1
7         for (int i = 0; i < n; ++i) f[i] = i;
8     }
9
10    int find(int x) { // 路徑壓縮
11        return f[x] == x ? x : (f[x] = find(f[x]));
12    }
13
14    bool isSameSet(int a, int b) {
15        return find(a) == find(b);
16    }
17
18    // 回傳 bool 有助於在 Kruskal 演算法中判斷是否成功加上一條邊，否則 void 即可
19    bool unite(int a, int b) {
20        int fa = find(a), fb = find(b);
21        if (fa == fb) return false;
22
23        // 小掛大優化：永遠讓 fa 是比較大的集合
24        if (sz[fa] < sz[fb]) swap(fa, fb);
25        sz[fa] += sz[fb];
26        f[fb] = fa;
27        return true;
28    }
29 };
30
31 // 概念：successor DSU，這個 disjoint set 將會是用可以來記錄 " 下一個位置 " 的
32 // set (f[i] 指向 find(i+1))

```

7.2 Kruskal 最小生成樹

用法：需搭配並查集使用。將所有邊排序後，利用併查集檢查是否形成環。

```

時間複雜度： $\mathcal{O}(E \log E)$ 
1 // 讓 Edge 支援比較大小，排序時會自動依據權重 w 由小到大排
2 struct Edge {
3     int u, v;
4     long long w;
5     bool operator<(const Edge& o) const {
6         return w < o.w;
7     }
8 };
9
10 struct Kruskal {
11     int n;
12     vector<Edge> edges;
13     vector<vector<pair<int, long long>>> tree_adj; // 用來存 MST 的樹狀圖，解
14     // DFS MST 用
15
16     void init(int _n) {
17         n = _n;
18         edges.clear();
19         tree_adj.assign(n + 1, vector<pair<int, long long>>());
20     }
21
22     void add_edge(int u, int v, long long w) {
23         edges.push_back({u, v, w});
24     }
25
26     // 回傳最小生成樹的總權重。若圖不連通則回傳 -1
27     long long solve() {
28         sort(edges.begin(), edges.end());
29         // 如果需要找 MaxST，請 sort(edges.rbegin(), edges.rend())，r!!
30
31         DSU dsu;
32         dsu.init(n + 1);
33
34         long long total_weight = 0;
35         int edge_cnt = 0; // 連接邊個數
36
37         for (auto& e : edges) {
38             // unite 若回傳 true，代表 u 和 v 原本不相連，合併成功 (無環)
39             if (dsu.unite(e.u, e.v)) {
40                 total_weight += e.w;
41                 edge_cnt++;
42
43                 // 成為 MST 的一部分，所以加入樹狀圖中
44                 tree_adj[e.u].push_back({e.v, e.w});
45                 tree_adj[e.v].push_back({e.u, e.w});
46
47                 if (edge_cnt == n - 1) break; // V 個點只要 V-1 條邊即完全連通
48             }
49         }

```

```

49         return (edge_cnt == n - 1) ? total_weight : -1;
50     }
51 }
52
53 // u: 當前點, target: 目標點, p: 父節點, cur_max: 目前最大邊權, 呼叫:
54 dfs_mst(st, ed, 0, 0)
55 long long dfs_mst(int u, int target, int p, long long cur_max) {
56     if (u == target) return cur_max; // 找到終點, 回傳整條路的瓶頸值
57
58     for (auto& edge : tree_adj[u]) {
59         int v = edge.first;
60         long long w = edge.second;
61
62         if (v != p) { // 不走回頭路, 完美省去 vis 陣列
63             // 帶著「目前遇過的最大邊權」繼續往下深搜
64             long long res = dfs_mst(v, target, u, max(cur_max, w));
65
66             // 如果 res 不是 -1, 代表在 v 的這條分支深處成功找到了 target
67             if (res != -1) return res;
68         }
69     }
70     return -1; // 這條死胡同找不到終點
71 }

```

7.3 次小生成樹

用法: 基於 Kruskal。會先求出最小生成樹, 再暴力枚舉拔除 MST 上的每一條邊, 找剩下的最小生成樹, 取最小值即為次小生成樹。

時間複雜度: $\mathcal{O}(V \times E)$

```

1 #include <vector>
2 #include <algorithm>
3 using namespace std;
4
5 const long long INF = 1e18;
6
7 // 1. 定義邊與 DSU
8 struct Edge {
9     int u, v;
10    long long w;
11    bool operator<(const Edge& rhs) const {
12        return w < rhs.w;
13    }
14 };
15
16 // struct DSU {}; 跟上面的一樣
17
18 int n, m; // 點數, 邊數
19 vector<Edge> edges;
20
21 // 2. 尋找次小生成樹
22 // 回傳值: pair< 最小生成樹權重, 次小生成樹權重 >
23 // 右圖不連通或不存在次小生成樹, 對應的權重會回傳 -1
24 pair<long long, long long> second_mst() {
25     sort(edges.begin(), edges.end()); // 權重由小到大排序
26
27     DSU dsu;
28     dsu.init(n);
29     long long mst_weight = 0;
30     int edge_cnt = 0;
31     vector<int> mst_edges; // 紀錄第一次 MST 用到的「邊的 index」
32
33     // Step 1: 求標準 MST 並記錄使用的邊
34     for (int i = 0; i < edges.size(); ++i) {
35         if (dsu.unite(edges[i].u, edges[i].v)) {
36             mst_weight += edges[i].w;
37             mst_edges.push_back(i);
38             edge_cnt++;
39         }
40     }
41
42     // 如果連第一次 MST 都構建不出來 (圖不連通)
43     if (edge_cnt != n - 1) return {-1, -1};
44
45     long long sec_mst_weight = INF;
46
47     // Step 2: 依序拔除 MST 上的每一條邊, 嘗試構造新的生成樹
48     for (int removed_idx : mst_edges) {
49         dsu.init(n); // 每次都要重置並查集
50         long long cur_weight = 0;
51         int cur_cnt = 0;
52
53         for (int i = 0; i < edges.size(); ++i) {
54             if (i == removed_idx) continue; // 【關鍵】禁用這條邊
55
56             if (dsu.unite(edges[i].u, edges[i].v)) {
57                 cur_weight += edges[i].w;
58                 cur_cnt++;
59             }
60         }
61
62         // 如果拔掉該邊後, 剩下的邊還是能連通所有點 (形成生成樹)
63         if (cur_cnt == n - 1) {
64             sec_mst_weight = min(sec_mst_weight, cur_weight);
65         }
66     }
67
68     if (sec_mst_weight == INF) sec_mst_weight = -1; // 不存在次小生成樹
69     return {mst_weight, sec_mst_weight};
70 }

```

7.4 最近共同祖先 LCA

用法: 使用倍增法, 利用二進位的特性, 預處理出每個節點向上跳 $2^0, 2^1, \dots, 2^i$ 步的祖先是誰

時間複雜度: 最差 $\mathcal{O}(N)$

```

1 const int MAXN = 2e5 + 5; // 依據題目給定的最大節點數量調整 MAXN
2 const int LOG = 19; // 2e5 的情況下 18+1=19 即可, 若數字超 2e5 則 LOG =
   __lg(MAXN) + 1
3
4 vector<int> adj[MAXN];
5 int up[MAXN][LOG];
6 int depth[MAXN];
7
8 // DFS 預處理深度與 2^i 祖先
9 void dfs(int u, int p) {
10     up[u][0] = p; // 2^0 的祖先就是直屬父親 p
11
12     for (int i = 1; i < LOG; ++i) { // 狀態轉移: 倍增建表
13         up[u][i] = up[up[u][i-1]][i-1];
14     }
15
16     for (int v : adj[u]) {
17         if (v != p) {
18             depth[v] = depth[u] + 1;
19             dfs(v, u);
20         }
21     }
22 }
23
24 int get_lca(int u, int v) { // 查詢最近公共祖先 (LCA)
25     if (depth[u] < depth[v]) swap(u, v); // 確保 u 是比較深的那個點
26
27     for (int i = LOG - 1; i >= 0; --i) { // 深度對齊
28         if (depth[u] - (1 << i) >= depth[v]) {
29             u = up[u][i];
30         }
31     }
32     if (u == v) return u;
33
34     for (int i = LOG - 1; i >= 0; --i) { // 同時往上跳, 尋找 LCA 的直屬子節點
35         if (up[u][i] != up[v][i]) {
36             u = up[u][i];
37             v = up[v][i];
38         }
39     }
40     return up[u][0];
41 }
42
43 int main() {
44     ios::sync_with_stdio(false);
45     cin.tie(0);
46
47     int n, q;
48     // 假設題目輸入為 n 個點, q 筆 LCA 查詢
49     if (!(cin >> n >> q)) return 0;
50
51     for (int i = 1; i <= n; ++i) adj[i].clear(); // 若題目有多筆測資, 必須初始化
52
53     for (int i = 0; i < n - 1; ++i) { // 讀取 n-1 條無向樹邊
54         int u, v;
55         cin >> u >> v;
56         adj[u].push_back(v);
57         adj[v].push_back(u);
58     }
59
60     depth[1] = 0; // 選擇根節點 (通常題目為 1, 或 0), 並初始化根節點的深度與
61     // DFS 預處理
62     dfs(1, 1); // 根節點的父親設為自己, 防止倍增跳躍時越界
63
64     while (q--) { // 處理 Q 筆查詢
65         int u, v;
66         cin >> u >> v;
67
68         int lca = get_lca(u, v);
69         cout << lca << "\n";
70
71         // 延伸: 若題目要求輸出兩點間的距離 (邊數)
72         // cout << depth[u] + depth[v] - 2 * depth[lca] << "\n";
73     }
74     return 0;
75 }

```

8 DP

DP 通用解題四步驟:

1. 定義陣列意義: 明確寫出 $dp[i]$ (或 $dp[i][j]$) 代表什麼。
2. 找出遞迴關係式: 推導出當前狀態如何由過去狀態轉移而來。
3. 設定邊界條件: 初始化起點或基礎 dp 值 (如: $dp[0] = 0$)。
4. 確認遍歷順序: 決定迴圈是從前到後 (Bottom-Up) 還是後到前 (Top-Down) 計算。(通常是 Bottom-Up, 思考“我是如何到這一步的”去推)

8.1 01 Knapsack 背包

用法: 解決物品不可拆分的情況, 若可拆分用 Greedy 拿比例最佳的部分

時間複雜度: $\mathcal{O}(N)$

```

1 /*
2 題目敘述: 給定 N 個物品的重量  $W_i$  和價值  $V_i$ , 和個容量為  $W$  的背包。選取若干件物品
   放入背包, 在不超過背包容量的情況下, 背包內物品價值總和最大為何?
3
4 定義  $dp[i][j]$  表示: 在考慮前 i 個物品, 且考慮重量  $\leq j$  的情況下, 所能得到的最大價
   值是多少 (假設 index 0 不放東西)
5 初始: 對於所有的  $i=0$ ,  $dp[i][j] = 0$  (不考慮任何物品的情況下, 最大價值是 0), 也可
   以直接預設所有位置都是 0
6 轉移函式:  $dp[i][j] = \max(dp[i-1][j], dp[i-1][j-W_i] + V_i)$ ;
7 -> 兩種情況:

```

```

8 1. 不放第 i 件物品，最大價值與前一項相同去
9 2. 放入第 i 件物品，最大價值為 扣除目標重量後的最大價值 加上當前物品的價值之總和
10 */
11
12 /*
13 ===== 原始 01 背包 =====
14 適用情況：需要 backtrack
15 -> 從 dp[0][0] 開始檢查：
16 如果 dp[i][j] == dp[i-1][j] 表示沒有拿第 i 個，接著讓 i = i-1 繼續跑
17 否則我們有拿第 i 個，接著跳到 dp[i-1][j-w[i]]
18 重複直到 i=0 (沒東西了) 或 j=0 (重量用完了)
19 */
20 int dp[0][0] = 0;
21 memset(dp, 0, sizeof(dp)); // 初始化為 0
22 for (int i = 1; i <= MAXN; ++i) // 從考慮第一個物品到第 N 個物品
23 {
24     for (int j = 0; j < w[i]; ++j) // 如果當前考慮重量小於 Wi，代表沒辦法裝下第
        i 個物品，直接繼承前一項
25         dp[i][j] = dp[i-1][j];
26     for (int j = w[i]; j <= MAXW; ++j) // 否則就用 transition function 找 max
27         dp[i][j] = max(dp[i-1][j], dp[i-1][j-w[i]] + v[i]);
28 }
29 cout << dp[0][MAXW] << "\n"; // 最終答案在最右下那格
30
31 /*
32 ===== 滾動陣列優化 =====
33 概念：在計算考慮第 i 個物品時，我們只會用到第 i-1 行的資料，可以看成陣列中的上一
        筆舊資料
34 透過想法上的極致壓縮，覆蓋無用資料空間，可將空間複雜度降至 O(W)
35 限制是 j 必須由大往小跑，如果由小至大會重複使用到陣列資料導致答案錯誤
36 */
37
38 int dp[0][MAXW];
39 memset(dp, 0, sizeof(dp));
40 for (int i = 1; i <= MAXN; ++i) // 注意一樣是要跑 MAXN 次，我們只是把陣列大小減
        少了
41     for (int j = MAXW; j >= w[i]; --j) // dp[j] 在等號左邊表示當前，等號右邊表
        示上一筆資料
42         dp[j] = max(dp[j], dp[j-w[i]] + v[i]); // 不需要考慮 j < w[i] 的情況，
        因為它會繼承上一個結果
43 cout << dp[0][MAXW] << "\n";
44
45 /*
46 ===== 完全背包 (無限背包) =====
47 已知：第 i 個物品重量為 wi，價值 vi；背包總容量 W，且每一個物品都可以重複 (無限
        次) 取用
48 */
49 int W;
50 int w[0][MAXN], v[0][MAXN];
51 int dp[0][MAXW];
52
53 memset(dp, 0, sizeof(dp));
54 for (int i = 1; i <= n; ++i)
55     for (int j = w[i]; j <= W; j++)
56         dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
57
58 /*
59 ===== 分組背包 =====
60 第 i 組第 j 個物品重量為 wij，價值 vij；背包總容量 W；但每組只能取一個
61 */
62 int W;
63 int dp[0][MAXW];
64 vector<vector<int>> w, v;
65
66 memset(dp, 0, sizeof(dp));
67 for (int i = 0; i < n; ++i)
68     for (int j = W; j >= 0; --j)
69         for (int k = 0; k < w[i].size(); ++k)
70             if (j >= w[i][k])
71                 dp[j] = max(dp[j], dp[j-w[i][k]] + v[i][k]);
72
73 /*
74 ===== 多背包共用商品 =====
75 題目：N 個物品 (同一項商品可被多個人同時取用)，G 個人各自有容量為 Wi 的背包，求所
        有人能拿的最大價值總和。
76
77 題解：每個人挑選互相獨立，不需重複計算。(查表)
78 1. 以所有商品跑一次標準一維 0/1 背包，每次讀入 Wi 與 Vi 時就 run 一次，建立 dp
        陣列。
79 2. 結束後跑一遍 dp[i] = max(dp[i], dp[i-1]) 確保單調性。
80 3. 讀入每個人的容量 W，總答案 ans += dp[W]。
81 */
82
83 // N 個物品，G 個人
84 // 假設最大可能的背包容量為 MAX_W
85 const int MAX_W = 100000;
86
87 int main() {
88     int N, G;
89     // 讀入物品數量與人數
90     if (!(cin >> N >> G)) return 0;
91
92     // dp[i] 代表容量為 i 的背包能裝的最大價值
93     vector<long long> dp(MAX_W + 1, 0);
94
95     // 1. 標準一維 0/1 背包
96     for (int i = 0; i < N; ++i) {
97         int w, v; // w: 重量, v: 價值
98         cin >> w >> v;
99         // 【注意】0/1 背包必須從大到小遍歷，避免重複拿取
100        for (int j = MAX_W; j >= w; --j) {
101            dp[j] = max(dp[j], dp[j-w] + v);
102        }
103    }
104
105    // 2. 確保單調遞增 (這步很關鍵，確保容量較大的人至少能拿到容量較小時的最大價
        值)
106    for (int j = 1; j <= MAX_W; ++j) {
107        dp[j] = max(dp[j], dp[j-1]);

```

```

108    }
109
110    // 3. 每個人各自查表，累加總價值
111    long long ans = 0;
112    for (int i = 0; i < G; ++i) {
113        int W; // 第 i 個人的背包容量
114        cin >> W;
115        ans += dp[W];
116    }
117
118    cout << ans << "\n";
119
120    return 0;
121 }
122
123 /*
124 === 恰好裝滿問題 / 子集和 ===
125 題目敘述：給定 N 樣物品，每件物品的重量分別為 Wi，問可不可以將物品分成等重的兩堆。
126
127 題解：首先判斷總重是否為二的倍數，如果不是就直接輸出 NO 然後下一個 case
128 如果可以的話就使用一維 DP 解，總重/2 當作 MAXW (dp 大小)，每件行李的重量是價值，
        看是否存在 dp[MAXW] = MAXW (在裝入一半行李的情況下，總重剛好也是一半)
129 */

```

8.2 LCS 最長共同子序列 (Sequence)

用法：二維 DP 與 Backtrack 找原始子序列

時間複雜度： $O(NM)$

```

1 // LCS (線性 DP) : dp[i][j] 代表的是 s1 的前 i 個字元，與 s2 的前 j 個字元。這是
        一個從索引 0 開始的前綴
2
3 struct LCS {
4     // 求解 LCS 並回傳最長共同子序列的字串
5     // 若只需長度，直接回傳 dp[n][m] 即可
6     string solve(const string& s1, const string& s2) {
7         int n = s1.length(), m = s2.length();
8
9         // dp[i][j] 代表 s1 的前 i 個字元與 s2 的前 j 個字元的 LCS 長度
10        // 宣告 n+1 與 m+1 是為了讓 index 0 作為「空字串」的邊界條件 (全為 0)
11        vector<vector<int>> dp(n + 1, vector<int>(m + 1, 0));
12
13        // 1. 狀態轉移 (填表)
14        for (int i = 1; i <= n; ++i) {
15            for (int j = 1; j <= m; ++j) {
16                if (s1[i-1] == s2[j-1]) {
17                    // 若字元相同，繼承左上角的答案 + 1
18                    dp[i][j] = dp[i-1][j-1] + 1;
19                } else {
20                    // 若字元不同，取上方或左方的最大值
21                    dp[i][j] = max(dp[i-1][j], dp[i][j-1]);
22                }
23            }
24        }
25
26        // 2. 回溯 (Backtracking) 找回原本的字串
27        string res = "";
28        int i = n, j = m;
29        while (i > 0 && j > 0) {
30            if (s1[i-1] == s2[j-1]) {
31                // 如果字元相同，這一定是 LCS 的一部分
32                res += s1[i-1];
33                i--; j--; // 往左上角回退
34            } else if (dp[i-1][j] > dp[i][j-1]) {
35                // 如果不同，往數值比較大的方向走 (此處為上方)
36                i--;
37            } else {
38                // 否則往左方走
39                j--;
40            }
41        }
42
43        // 因為是從尾巴推回去的，最後要把字串反轉
44        reverse(res.begin(), res.end());
45        return res;
46    }
47 };
48
49 /*
50 變化：給定兩個字串，可以任意進行插入、刪除、取代三種操作
51 請問最少需要幾次操作可以使得兩字串完全一模一樣，並輸出過程？
52 與 LCS 大致相同，(相同、選左、選上)，但是多了一個取代的操作
53 所以轉移方程式會變成
54 dp[i][j] = min{
55     1. if (x[i] == y[j]) dp[i-1][j-1] 字元相同的情況
56     2. if (x[i] != y[j]) dp[i-1][j-1]+1 選擇"取代"
57     3. dp[i-1][j]+1 選擇"左邊"
58     4. dp[i][j-1]+1 選擇"右邊"
59 }
60
61 補充：
62 Longest Palindrome Sequence 最長迴文子序列
63 給定一個字串，問你該字串的最長迴文子序列 "長度" 為多少
64 做法就是對 s1 與 s2=reverse(s1) 做一次 LCS，即可得到 "最長長度"
65
66 變化：求最少插入/刪除次數以形成迴文
67 如果只求長度 -> strlen - LPS(s)
68 否則就要使用到區間 DP 的內容了
69 */

```

8.3 LIS / LDS 最長遞增/遞減子序列

用法：嚴格遞增用 lower_bound，非嚴格改用 upper_bound。替換策略下的 dp 陣列長度正確，但內容不一定是真實子序列。

時間複雜度： $O(N \log N)$

```

1 /*
2 二維套疊問題 (Russian Doll Envelopes / 箱子堆疊)
3 題意：給你一堆信封，每個有寬度和高度，問你最多可以把幾個信封像俄羅斯娃娃一樣套在一起
4 解法：將寬度由小到大排序；當寬度相同時，高度必須由大到小排。接著，直接對高度的陣列求 LIS
5
6 橋樑/連線不交叉問題 (Building Bridges)
7 題意：南岸和北岸各有 N 座城市，題目給定連線配對。要求建最多座橋，且橋與橋之間不能交叉
8 解法：把南岸的城市編號由小到大排序，然後對它們對應的北岸城市編號求 LIS
9
10 最少覆蓋路徑 / 導彈攔截系統 (Dilworth 定理)
11 題意：系統每次只能攔截「高度遞減」的飛彈。給你一連串飛彈的高度，問你「最少」需要裝備幾套攔截系統？
12 解法：「最少需要的遞減子序列數量」等價於「最長遞增子序列 (LIS) 的長度」。求出 LIS 長度就直接是答案！
13
14 最長雙坡子序列 (Bitonic Subsequence)
15 題意：求一個數列，先嚴格遞增再嚴格遞減的最長長度。
16 解法：從左到右跑一次 LIS，記錄每個位置結尾的 LIS 長度；再從右到左跑一次 LIS (相當於 LDS)，記錄每個位置開頭的 LDS 長度。兩者相加減 1 的最大值即為所求。
17 */
18
19 // 1. 最長遞增子序列 (LIS) - (嚴格) 遞增
20 int get_lis(const vector<int>& arr) {
21     vector<int> dp;
22     for (int x : arr) {
23         // lower_bound: 找第一個 >= x 的位置 (若要求非嚴格遞增，改用 upper_bound)
24         auto it = lower_bound(dp.begin(), dp.end(), x);
25         if (it == dp.end()) {
26             dp.push_back(x); // 比所有數字都大，序列長度 + 1
27         } else {
28             *it = x; // 貪心策略：替換成較小的結尾，增加後續連接的潛力
29         }
30     }
31     return dp.size(); // dp 陣列的長度就是 LIS 的長度 (注意：dp 內容不一定是真實 LIS)
32 }
33
34 // 2. 最長遞減子序列 (LDS) - (嚴格) 遞減
35 int get_lds(const vector<int>& arr) {
36     vector<int> dp;
37     for (int x : arr) {
38         // LDS 只需要搭配 greater<int>() 來找第一個 <= x 的位置
39         // 若要求非嚴格遞減，改用 less_equal<int>() 等自訂比較
40         auto it = lower_bound(dp.begin(), dp.end(), x, greater<int>());
41         if (it == dp.end()) {
42             dp.push_back(x);
43         } else {
44             *it = x;
45         }
46     }
47     return dp.size();
48 }

```

8.4 LCS 轉 LIS

用法：兩個序列的「公共子序列」，在序列 A 中的索引必定是嚴格遞增的，所以可以將 B 中的每一個數字替換成他在 A 中的位置 (索引)，接著對新陣列求 LIS，長度就會剛好等於 LCS

時間複雜度： $\mathcal{O}(K \log K)$ K 為元素配對數量

```

1 vector<int> lcs_to_lis(const vector<int>& A, const vector<int>& B) {
2     unordered_map<int, vector<int>> pos_A;
3
4     // 1. 紀錄序列 A 中每個元素出現的所有索引
5     for (int i = 0; i < A.size(); ++i) {
6         pos_A[A[i]].push_back(i);
7     }
8
9     vector<int> transformed;
10
11     // 2. 走訪序列 B，將其替換為 A 中的索引
12     for (int x : B) {
13         if (pos_A.count(x)) {
14             // 重點：必須從大到小 (反向) 加入索引！
15             // 避免 B 中的同一個元素，匹配到 A 中的多個相同元素。
16             for (int i = pos_A[x].size() - 1; i >= 0; --i) {
17                 transformed.push_back(pos_A[x][i]);
18             }
19         }
20     }
21
22     // 3. 找出轉換後的 LIS 陣列
23     vector<int> lis;
24     for (int x : transformed) {
25         auto it = lower_bound(lis.begin(), lis.end(), x);
26         if (it == lis.end()) {
27             lis.push_back(x);
28         } else {
29             *it = x; // 貪心策略：替換掉第一個大於等於 x 的數字
30         }
31     }
32
33     return lis; // LIS 長度與 LCS 一致，可直接 return lis.size(); 就是 LCS 的長度
34 }
35
36 /*
37 如果要對 LIS 序列找 LCS：
38 1. A 為原序列，B 為 sort(A)
39 2. 直接對 A、B 做 LCS
40 */

```

8.5 區間 DP 概念 (最長迴文子序列)

用法：區間 DP 是由字串的內部向外擴張，利用已經算好的較短子字串，來推導出較長字字串的答案。

時間複雜度： $\mathcal{O}(N^2)$

```

1 // LPS (區間 DP) : dp[i][j] 代表的是同一個字串 s 裡面，從索引 i 到 j 的這段開區
   // 間 (也就是子字串 s[i...j])
2
3 struct IntervalDP {
4     // 求解最長迴文子序列 (LPS) 長度
5     int solveLPS(const string& s) {
6         int n = s.length();
7         if (n == 0) return 0;
8
9         // dp[i][j] 代表字串 s 區間 [i, j] 內的最長迴文子序列長度
10        vector<vector<int>> dp(n, vector<int>(n, 0));
11
12        // 1. 狀態轉移 (填表)
13        // 注意 i 是從字串的「尾巴」往「頭」掃
14        for (int i = n - 1; i >= 0; --i) {
15            // Base case: 單一字元本身就是長度為 1 的迴文
16            dp[i][i] = 1;
17
18            // j 從 i 的下一個位置開始向右擴展
19            for (int j = i + 1; j < n; ++j) {
20                if (s[i] == s[j]) {
21                    // 若頭尾字元相同，長度為內部區間的迴文長度 + 2
22                    // 因為 i 是倒著掃，j 是正著掃，所以 dp[i+1][j-1] 絕對已經算過了
23                    dp[i][j] = dp[i + 1][j - 1] + 2;
24                } else {
25                    // 若頭尾不同，看是捨棄左邊 (s[i]) 還是捨棄右邊 (s[j]) 比較長
26                    dp[i][j] = max(dp[i + 1][j], dp[i][j - 1]);
27                }
28            }
29        }
30
31        // 最終答案就是涵蓋整個字串的區間 [0, n-1]
32        return dp[0][n - 1];
33    }
34
35    // 求解變成迴文的最少編輯距離 (包含插入、刪除、替換)
36    // 如果求最少 插入/刪除 字元，才能讓字串變成迴文的題目，請用 strlen - LPS(s)
37    pair<int, string> solvePalindromicDistance(const string& s) {
38        // 1. 找最少編輯次數
39        int n = s.length();
40        if (n == 0) return {0, ""};
41
42        // dp[i][j] 代表將字串 s 區間 [i, j] 變成迴文的最少編輯次數
43        vector<vector<int>> dp(n, vector<int>(n, 0));
44
45        // i 從字串的「尾巴」往「頭」掃
46        for (int i = n - 1; i >= 0; --i) {
47            dp[i][i] = 0; // 單一字元本身就是迴文，編輯距離為 0
48
49            // j 從 i 的下一個位置開始向右擴展
50            for (int j = i + 1; j < n; ++j) {
51                if (s[i] == s[j]) {
52                    // 頭尾相同，不需要編輯
53                    // 當 j == i+1 時，dp[i+1][j-1] 即 dp[i+1][i] 會是 0 (因為預設值)
54                    dp[i][j] = dp[i + 1][j - 1];
55                } else {
56                    // 頭尾不同，取三種操作的最小值 + 1
57                    // 1. dp[i+1][j] -> 插入/刪除對應 s[i]
58                    // 2. dp[i][j-1] -> 插入/刪除對應 s[j]
59                    // 3. dp[i+1][j-1] -> 直接把 s[i] 替換成 s[j] 或反之
60                    dp[i][j] = min({dp[i + 1][j], dp[i][j - 1], dp[i + 1][j - 1]}) + 1; // 如果有定義 cost 要記得改
61                }
62            }
63        }
64
65        // 2. 狀態回溯 (Backtracking)
66        string left_part = "", right_part = "";
67        int i = 0, j = n - 1;
68
69        while (i < j) {
70            if (s[i] == s[j]) { // 若頭尾相同，直接加入答案
71                left_part += s[i];
72                right_part += s[j];
73                i++; j--;
74            } else {
75                // 若頭尾不同，檢查是哪條最佳路徑轉移過來的
76                bool can_sub = (dp[i + 1][j - 1] + 1 == dp[i][j]); // 選擇用替代的
77                bool can_del_i = (dp[i + 1][j] + 1 == dp[i][j]); // 選擇刪除 i 的
78                bool can_del_j = (dp[i][j - 1] + 1 == dp[i][j]); // 選擇刪除 j 的
79
80                int best_c = 256; // 用來尋找字典序最小的字元 (ASCII 上限)
81                int choice = -1; // 1: 替換, 2: 處理左側, 3: 處理右側
82
83                // 評估替換策略 (選擇兩者中較小的字元)
84                if (can_sub) {
85                    char c = min(s[i], s[j]);
86                    if (c < best_c) { best_c = c; choice = 1; }
87                }
88                // 評估處理左側策略 (對應 s[i])
89                if (can_del_i) {
90                    char c = s[i];
91                    if (c < best_c) { best_c = c; choice = 2; }
92                }
93                // 評估處理右側策略 (對應 s[j])
94                if (can_del_j) {
95                    char c = s[j];
96                    if (c < best_c) { best_c = c; choice = 3; }
97                }

```

```

97         }
98
99         // 寫入最佳字元
100         left_part += (char)best_c;
101         right_part += (char)best_c;
102
103         // 根據最佳策略移動指標
104         if (choice == 1) { i++; j--; }
105         else if (choice == 2) { i++; }
106         else if (choice == 3) { j--; }
107     }
108 }
109
110 // 若最後指標交會於同一個字元，該字元放正中間
111 if (i == j) {
112     left_part += s[i];
113 }
114
115 // 組合最終字串（右半部需要反轉）
116 reverse(right_part.begin(), right_part.end());
117
118 return {dp[0][n-1], left_part + right_part};
119 }
120 };
121
122 /*
123 有關迴文的其他題目補充
124 變化 1: 計算迴文字序列的「總數」（排容原理）
125 if(s[i] != s[j]) dp[i][j] = dp[i+1][j] + dp[i][j-1] - dp[i+1][j-1]
126 if(s[i] == s[j]) dp[i][j] = dp[i+1][j] + dp[i][j-1] + 1
127
128 變化 2: 跨字串（左 A 右 B）的迴文
129 令 dp[i][j] 表示「A 從索引 i 開始的後綴」與「B 從索引 j 開始的前綴」所能構成的最
    長條件迴文長度。
130 if(A[i] == B[j]) dp[i][j] = dp[i+1][j-1] + 2
131 if(A[i] != B[j]) dp[i][j] = max(dp[i+1][j], dp[i][j-1])
132 */

```

8.6 跳格子

用法：給定一組陣列，固定跳躍距離，問你最少需要跳幾次，以及所跳過的格子 index。如果數值達到 long long，INF 需設為 1e18 或 0x3f3f3f3f3f3f3f3f

時間複雜度： $\mathcal{O}(N \times K)$

```

1 //
2 題目出處：PUPC 2025 PC River Crossing
3 題目敘述：給定一組陣列，1 表示可達，0 表示不可達，問你最少需要跳幾次可以跳到終點，
    以及跳時經過的路徑
4 如果不需要輸出路徑，可以直接用 greedy，但要輸出路徑就需要用 DP
5 */
6
7 #include <bits/stdc++.h>
8 using namespace std;
9
10 int main(){
11     int kase;
12     cin >> kase;
13     while(kase--){
14         int n;
15         cin >> n;
16         vector<int> v(n), ans(n, 0x3f3f3f3f), pre(n, -1);
17         // v 存原始陣列，ans 存抵達該格的最短步數，pre 存 在最短步數的情況下，我上一格走
            的位置
18         for(int i=0; i<n; ++i) cin >> v[i]; // 讀入陣列
19         ans[0] = 0; // 初始化：跳到第 0 格的距離是 0
20         for(int i=1; i<n; ++i){ // 接下來每次遍例我的前三格
21             if(v[i] == 0) continue; // 如果該格不可達，
22             for(int j=i; j<=3; ++j){ // 這裡的 3 是題目設定的，請改成該題目設置的數值
23                 if(i-j >= 0 && v[i-j] == 1 && ans[i-j] != 0x3f3f3f3f){
24                     // 判斷：1. 在邊界內，2. 該格可走，3. 重複確認該格可走（如不可走，ans[i-j] 會
                        直接 skip，保留 INF）
25                     if(ans[i] >= ans[i-j] + 1){ // 注意題目要求：相同距離的情況下，格子要
                            選哪一個，在這題中紀錄的格子要越小越好
26                         ans[i] = ans[i-j] + 1;
27                         pre[i] = i-j;
28                     }
29                 }
30             }
31         }
32         if(ans[n-1] == 0x3f3f3f3f) cout << -1 << '\n';
33         else{
34             cout << ans[n-1] << '\n';
35
36             int i = n-1;
37             stack<int> st;
38             st.push(n-1);
39             while(i > 0){
40                 st.push(pre[i]);
41                 i = pre[i];
42             }
43
44             bool first = true;
45             while(!st.empty()){
46                 if(!first) cout << ' ';
47                 first = false;
48                 cout << st.top();
49                 st.pop();
50             }
51             cout << '\n';
52         }
53     }
54     return 0;
55 }

```

8.7 分錢幣問題

用法：完全背包變體。求「方法數」需外層枚舉硬幣、內層枚舉金額（求組合數）。求「最少硬幣數」轉移取 min。於 main 開頭先建表。

時間複雜度： $\mathcal{O}(\text{硬幣數} \times \text{最大金額})$

```

1 #include <iostream>
2 #include <vector>
3 using namespace std;
4
5 // 題型：給定數種硬幣，求「湊出金額 N 有幾種方法？」（例如 UVA 357）
6 // 注意：方法數通常成長得非常快，必定會超過 int，請一律使用 long long！
7
8 const int MAX_AMOUNT = 30005; // 依據題目給定的最大金額修改
9 long long dp[MAX_AMOUNT];
10 int coins[] = {1, 5, 10, 25, 50}; // 依據題目給定的硬幣種類修改
11
12 // 預處理建表：在 main() 迴圈開始前呼叫一次即可
13 void build_coin_change() {
14     // Base case: 湊出金額 0 的方法只有 1 種（就是所有硬幣都不拿）
15     dp[0] = 1;
16
17     // 【核心陷阱：必須先枚舉硬幣，再枚舉金額】
18     // 這樣求出來的才是「組合數」（1+5 和 5+1 視為同一種方法）
19     for (int coin : coins) {
20         // 從該硬幣的面額開始往上推，因為金額小於硬幣面額時根本用不到這枚硬幣
21         for (int j = coin; j < MAX_AMOUNT; ++j) {
22             dp[j] += dp[j - coin];
23         }
24     }
25 }
26
27 /* * 延伸變體：如果題目是求「湊出金額 N 最少需要幾枚硬幣？」
28 * 1. 將 dp 陣列初始化為 INF，dp[0] = 0。
29 * 2. 轉移方程式改為：dp[j] = min(dp[j], dp[j - coin] + 1);
30 */

```

8.8 Kadane's Algorithm (最大子陣列和)

用法：尋找陣列中連續元素相加的最大值。若全為負數會正確回傳最大的負數。

時間複雜度： $\mathcal{O}(N)$

```

1 // 應用場景：題目要求在一個陣列中，找出一小段「連續」的數字，使得它們的總和最大。
2 long long kadane(const vector<long long>& arr) {
3     long long best = -1e18; // 紀錄歷史最大值（預設極小值以防全負數）
4     long long curr_max = 0; // 紀錄加上當前數字後的局部最大值
5
6     for (long long x : arr) {
7         // 核心精神：如果加上自己，比自己單獨一個還要小，那不如「從自己重新開始」
8         curr_max = max(x, curr_max + x); // x 是新的值
9         best = max(best, curr_max);
10    }
11    return best;
12 }

```

9 Greedy

9.1 一維區間全覆蓋

用法：需要先把輸入轉化成一個包含 l, r 的 struct，自訂 operator<，起始位置由小到大排序

時間複雜度： $\mathcal{O}(N \log N)$

```

1 //
2 題目敘述：給定 N 個灑水器，每個灑水器包含座標 P 與半徑 R，已知草坪長度為 L，寬度
    為 W，請問最少需要開啟幾個灑水器就能覆蓋整個區間（或指定哪些灑水器）
3
4 題解：先將灑水器圍轉化為一個 struct point，用 l, r 區間表示能覆蓋的左右範圍，然
    後自訂 operator<
5 1. 用開始位置排序
6 2. 每次都在 小於當前右界的區間中，選擇能往右最多的那個
7 3. 重複直到右區間 <L，得到最少數量；或者是在當前右邊界內找不到可選擇的表示不存在
    解
8 */
9 const double eps = 1e-8;
10 struct point{
11     double L, R;
12     bool operator<(const point&o) const{
13         return L < o.L;
14     }
15 };
16
17 vector<point> v;
18 double pos, rad;
19 for(int i=0; i<n; ++i){
20     cin >> pos >> rad;
21     double real;
22     if(rad - w/2.0 < eps) continue; // 判斷如果半徑小於寬度就不可能被選，直接
        不加入
23     real = sqrt(rad*rad - (w/2.0)*(w/2.0)); // real 表示實際可覆蓋距離
24     v.push_back({pos-real, pos+real});
25 }
26 sort(v.begin(), v.end());
27
28 int cnt = 0; // 總計數，如果要指定哪些灑水器就用 vector 存每次加入的
29 int i = 0;
30 double furthest = 0.0;
31 bool noAns = false;
32
33 while(L - furthest > eps){ // L > furthest
34     double cur = furthest;
35     while(i < v.size() && v[i].L - furthest < eps){
36         cur = max(cur, v[i].R);
37         ++i;
38     }
39     if(cur - furthest < eps){
40         noAns = true;
41         break;
42     }
43     furthest = cur;
44     ++cnt;
45 }
46 if(noAns) cout << -1 << '\n';
47 else cout << cnt << '\n';

```

9.2 間格跳躍

用法: 給定一組陣列, 距離為 `index` 的值, 問你最少需要跳多少次才能到終點

時間複雜度: $\mathcal{O}(N)$

```
1 int jump(vector<int>& nums) {
2     int jumps = 0; // 總跳躍次數
3     int current_end = 0; // 「當前這一步」能到達的最遠邊界
4     int farthest = 0; // 探索過程中, 發現「下一步」能到達的最遠距離
5     bool noAns = false;
6
7     // 注意: 迴圈只走到 nums.size() - 2
8     // 因為如果已經站在最後一個位置, 就不需要再起跳了
9     for (int i = 0; i < nums.size() - 1; ++i) {
10        // 邊走邊看, 更新下一跳的最遠潛力
11        farthest = max(farthest, i + nums[i]);
12
13        // 當走到了「當前這步」的極限邊界時, 就必須結算, 進行「下一跳」
14        if (i == current_end) {
15            if (current_end == farthest) { // 如果被迫跳但發現一步都沒跳就代表走不到終點
16                noAns = true;
17                break;
18            }
19            jumps++;
20            current_end = farthest; // 將邊界擴展到剛才探索到的最遠距離
21
22            if (current_end >= nums.size() - 1) break; // 提前結束的優化
23        }
24    }
25    if (noAns) return -1;
26    return jumps;
27 }
```

9.3 最大不重疊區間

用法: 對一組任務做排程, 請問最多可以移除多少個任務 -> 最早結束的先選。

時間複雜度: $\mathcal{O}(N \log N)$

```
1 int countRemoves(vector<pair<int, int>> &intervals) {
2     if (intervals.empty()) return 0; // 不需要移除任何東西
3
4     // 用結束時間做排序
5     sort(intervals.begin(), intervals.end(), [](const auto& a, const auto& b) { return a.second < b.second; });
6
7     int removed = 0;
8     int cur_end = intervals[0].second; // 一開始必定選第一個任務
9     for (int i = 1; i < intervals.size(); ++i) { // 遍歷所有任務
10        if (cur_end > intervals[i].first) ++removed; // 如果我目前的 end 能覆蓋到當前 i 的左邊界就移除 i
11        else cur_end = intervals[i].second; // 否則我就要選 i, 並設為當前右邊界
12    }
13
14    return removed;
15 }
```

9.4 最小覆蓋區間

用法: 給定一組陣列, 包含開始時間與結束時間, 求最少需要幾個房間就能完成所有任務

時間複雜度: $\mathcal{O}(N \log N)$

```
1 struct Interval {
2     int st, ed;
3     bool operator<(const Interval &o) const {
4         if (st != o.st) return st < o.st; // 先以開始時間排序
5         return ed < o.ed; // 開始時間相同, 較早結束的排前面
6     }
7 };
8
9 int minMeetingRooms(vector<Interval>& intervals) {
10    if (intervals.empty()) return 0;
11
12    // 1. 依照開始時間排序
13    sort(intervals.begin(), intervals.end());
14
15    // 2. 建立一個 Min-Heap, 專門用來儲存「使用中會議室的結束時間」
16    // !! 如果需要印出順序, 只要把 int(結束時間) 改成 pair<int, int>(結束時間, 房間編號)!!
17    priority_queue<int, vector<int>, greater<int>> min_heap;
18
19    // 3. 掃描每一個會議
20    for (const auto& interval : intervals) {
21        // 如果 pq 不為空, 且最早結束的會議已經可以使用了 (當前會議的開始時間 >= 最早空出來的房間時間)
22        if (!min_heap.empty() && interval.st >= min_heap.top()) {
23            min_heap.pop(); // 該會議室空出來了, 把舊的結束時間 pop 掉 (準備重複利用)
24        }
25
26        // 把當前會議的結束時間塞進去 (代表佔用了一間會議室)
27        // 不管是開了一間新房間, 還是接續前一間, 當前的結束時間都要記錄
28        min_heap.push(interval.ed);
29    }
30
31    return min_heap.size(); // 最終 Heap 裡面剩下幾個元素, 就代表我們同時需要幾個會議室
32 }
```

9.5 區間選點問題

用法: 將區間依照右界由小到大排序。遍歷所有區段, 如果當前區間的左界大於目前選定點的右界, 代表必須多選一個點

時間複雜度: $\mathcal{O}(N \log N)$

```
1 struct Interval {
2     int l, r;
3     // 依右界由小到大排序, 右界相同時左界小的優先
4     bool operator<(const Interval& o) const {
5         if (r != o.r) return r < o.r;
6         return l < o.l;
7     }
8 };
9
10 int minPoints(vector<Interval>& intervals) {
11     if (intervals.empty()) return 0;
12     sort(intervals.begin(), intervals.end());
13
14     int cnt = 1;
15     int current_r = intervals[0].r;
16
17     for (int i = 1; i < intervals.size(); ++i) {
18        // 若當前區間左界 > 目前紀錄的右界, 代表無法共用同一個點
19        if (intervals[i].l > current_r) {
20            cnt++;
21            current_r = intervals[i].r; // 更新右界
22        }
23    }
24    return cnt;
25 }
```

9.6 最小化最大延遲問題

用法: 按照到期時間從早到晚處理。記錄當下累積的耗時, 並與該任務的死線相減, 取最大的延遲時間

時間複雜度: $\mathcal{O}(N \log N)$

```
1 struct Job {
2     int time, deadline;
3     // 依到期時間由早到晚排序
4     bool operator<(const Job& o) const {
5         return deadline < o.deadline;
6     }
7 };
8
9 int minimizeLateness(vector<Job>& jobs) {
10    sort(jobs.begin(), jobs.end());
11
12    int current_time = 0;
13    int max_lateness = 0;
14
15    for (const auto& job : jobs) {
16        current_time += job.time;
17        // 延遲時間 = 完成時間 - 到期時間。若提早完成則延遲為 0
18        int lateness = max(0, current_time - job.deadline);
19        max_lateness = max(max_lateness, lateness);
20    }
21    return max_lateness;
22 }
```

9.7 Huffman 樹

用法: 每次合併兩堆, 成本是兩堆的重量和, 求最小總成本

時間複雜度: $\mathcal{O}(N \log N)$

```
1 long long huffmanCost(const vector<int>& weights) {
2     // 宣告 Min-Heap
3     priority_queue<long long, vector<long long>, greater<long long>> pq;
4     for (int w : weights) pq.push(w);
5
6     long long total_cost = 0;
7
8     // 每次挑選最小的兩堆合併, 直到只剩一堆
9     while (pq.size() > 1) {
10        long long first = pq.top(); pq.pop();
11        long long second = pq.top(); pq.pop();
12
13        long long merged = first + second;
14        total_cost += merged;
15        pq.push(merged); // 將合併後的新重量塞回 Heap
16    }
17
18    return total_cost;
19 }
```

9.8 任務調度問題

用法: 依照懲罰由大到小排序, 每項工作依序嘗試可不可以放在 $D_i - T_i + 1, D_i - T_i, \dots, 1, 0$, 如果有空間就放進去, 否則延後執行。僅限 $N_i \cdot \text{deadline}$ 10^6 , 如果更大須使用 Greedy + MinHeap, 只以 `deadline` 遞增排序, 接著走訪所有任務的同時, 將懲罰值 `push` 進 `heap`。如果當前 `Heap` 的大小大於當前任務的 `deadline`, 代表你排了太多任務會逾期。這時直接把 `Heap` 裡面懲罰值最小的那個 `pop` 掉。

時間複雜度: $\mathcal{O}(N)$

```
1 struct Task {
2     int deadline, penalty;
3     // 依懲罰由大到小排序
4     bool operator<(const Task& o) const {
5         return penalty > o.penalty;
6     }
7 };
8
9 // 利用並查集快速找尋「最晚可用的空間天數」
10 struct DSU {
11     vector<int> parent;
```

```

12 DSU(int n) {
13     parent.resize(n + 1);
14     for (int i = 0; i <= n; ++i) parent[i] = i;
15 }
16 int find(int x) {
17     return parent[x] == x ? x : (parent[x] = find(parent[x]));
18 }
19 };
20
21 long long taskScheduling(vector<Task>& tasks) {
22     sort(tasks.begin(), tasks.end());
23
24     int max_deadline = 0;
25     for (const auto& t : tasks) {
26         max_deadline = max(max_deadline, t.deadline);
27     }
28
29     DSU dsu(max_deadline);
30     long long total_penalty = 0;
31
32     for (const auto& t : tasks) {
33         // 尋找此任務到期日之前，最晚可用的空檔
34         int available_slot = dsu.find(t.deadline);
35
36         if (available_slot > 0) {
37             // 有空位：佔用這天，並將這天的 parent 指向前一天
38             dsu.parent[available_slot] = dsu.find(available_slot - 1);
39         } else {
40             // 沒空位：只能放棄並承受懲罰
41             total_penalty += t.penalty;
42         }
43     }
44     return total_penalty;
45 }

```

9.9 多機調度問題

用法：將工作依耗時「由大到小」排序。每次將工作交給最快空閒出來的機器。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 struct Machine {
2     int id;
3     long long completion_time;
4     // Min-Heap：優先選擇最快空閒的機器。若時間相同，優先給 ID 小的
5     bool operator>(const Machine& o) const {
6         if (completion_time != o.completion_time)
7             return completion_time > o.completion_time;
8         return id > o.id;
9     }
10 };
11
12 long long multiprocessorScheduling(vector<int>& jobs, int m) {
13     // 工作依耗時由大到小排序 (Longest Processing Time first)
14     sort(jobs.begin(), jobs.end(), greater<int>());
15
16     priority_queue<Machine, vector<Machine>, greater<Machine>> pq;
17     for (int i = 0; i < m; ++i) pq.push({i, 0});
18
19     // 依序把工作派給最快結束的機器
20     for (int job_time : jobs) {
21         Machine earliest = pq.top();
22         pq.pop();
23
24         earliest.completion_time += job_time;
25         pq.push(earliest);
26     }
27
28     // 找出所有機器中最晚完工的時間
29     long long max_time = 0;
30     while (!pq.empty()) {
31         max_time = max(max_time, pq.top().completion_time);
32         pq.pop();
33     }
34
35     return max_time;
36 }

```

9.10 最大區間重疊數 Sweep Line

用法：詢問在任意時刻可達到的最大重疊數量為何。將區間拆分為 +1 (開始) 與 -1 (結束) 事件。時間相同時 -1 優先處理，避免多算重疊數。

時間複雜度： $\mathcal{O}(N \log N)$

```

1 // 解決「最大區間重疊數」(簡易版 Sweep Line)
2 int sweep_line_overlap(const vector<pair<int, int>>& intervals) {
3     // 儲存所有事件：pair<時間點, 變化量>
4     vector<pair<int, int>> events;
5
6     for (auto& interval : intervals) {
7         events.push_back({interval.first, 1}); // 開始時間，需求 +1
8         events.push_back({interval.second, -1}); // 結束時間，需求 -1
9     }
10
11     // 排序規則：先按時間由小到大排；
12     // 若時間相同，變化量小的 (-1) 會自動排在 (+1) 前面！
13     // 這確保了「先退房間，再借房間」，不產生多餘的重疊。
14     sort(events.begin(), events.end());
15
16     int current_overlap = 0;
17     int max_overlap = 0;
18
19     for (auto& event : events) {
20         current_overlap += event.second;
21         max_overlap = max(max_overlap, current_overlap);
22     }
23
24     return max_overlap;
25 }

```

9.11 Josephus 約瑟夫問題

用法：詢問約瑟夫問題的最後贏家 (存活者)

時間複雜度： $\mathcal{O}(K \log N)$

```

1 // JosephusProblem，只是規定要先砍 1 號
2 // 所以當作有 n - 1 個人，目標的 13 號移成 12
3 // 再者從 0 開始比較好算，所以目標 12 順移成 11
4
5 // 0(n)
6 int getWinner(int n, int k) {
7     int winner = 0;
8     for (int i = 1; i <= n; ++i)
9         winner = (winner + k) % i;
10    return winner;
11 }
12
13 int main() {
14     int n;
15     while (cin >> n && n) {
16         --n; // 因為要用 初始 idx 為 0
17         for (int k = 1; k <= n; ++k) {
18             if (getWinner(n, k) == 11) { // 依據目標進行修改
19                 printf("%d\n", k);
20                 break;
21             }
22         }
23     }
24
25     return 0;
26 }
27
28 // O(k log(n))
29 int josephus(int n, int k) {
30     if (n == 1) return 0;
31     if (k == 1) return n - 1;
32     if (k > n) return (josephus(n - 1, k) + k) % n;
33
34     int res = josephus(n - n / k, k);
35     res -= n % k;
36     if (res < 0) res += n; // 依據需求 mod n
37     else res += res / (k - 1); // 還原位置
38
39     return res;
40 }

```

10 Search & Techniques

10.1 Binary Search

用法：二分搜技巧

時間複雜度： $\mathcal{O}(\log N)$

```

1 Binary Search = 找答案，但不一定是對陣列二分搜
2
3 // === 基本找值 ===
4 // 需要在 sorted array
5 while(l <= r){
6     int mid = l + (r-l)/2; // 避免 overflow
7     if(mid == target) return mid;
8     else if(mid < target) l = mid+1; // mid 在 target 左邊
9     else r = mid-1;
10 }
11 // 可以直接用 lower_bound(st, ed, target) 找位置
12
13 // === 對答案二分搜 ===
14 /*
15 找「最大值的最小值 Minimax」或「最小值的最大值 Maximin」
16 要求答案具有單調性 即 X 合法，則 <= 或 >= X 者都合法，常見於構造題
17
18 例題：給定長度為 N 的前綴和正負號狀態 (+, -, 0)，求構造符合狀態且元素不含 0 的陣
19 列內元素最大絕對值的最小值。
20 如：+0+- 最小 cost 為 2，正確 array 為：[2, -2, 2, -1, -2]，而非 [1, -1, 1, 1, -3]
21 */
22 bool check(int M, int n, const string& s) { // 步長為 M
23     long long L = 0, R = 0; // 答案可達到的左 (低)、右 (高) 邊界
24
25     for (int i = 0; i < n; ++i) {
26         char c = s[i];
27         long long prevL = L, prevR = R;
28
29         L = L - M;
30         R = R + M;
31
32         if (c == '+') L = max(L, 1LL); // 要求 +，左邊界至少要是 1
33         else if (c == '-') R = min(R, -1LL); // 要求 -，右邊界至少要是 -1
34         else if (c == '0') { // 要求 0，左右邊界都必須 0
35             L = max(L, 0LL);
36             R = min(R, 0LL);
37         }
38
39         if (M == 1) {
40             if (abs(L) % 2 != (i + 1) % 2) L++;
41             if (abs(R) % 2 != (i + 1) % 2) R--;
42         } else {
43             if (prevL == prevR && L == prevL && R == prevR) { // L, R 因為 M
44                 // 不夠大，被迫不變 (a 為 0)
45                 return false;
46             }
47         }
48     }
49     if (L > R) return false; // 如果左邊界大於右邊界則這個 M 無法成功
50     return true;
51 }
52 void solve() {
53     int n;
54     cin >> n;

```

```

55     string s;
56     cin >> s;
57
58     int left = 1, right = n, ans = -1;
59
60     while (left <= right) {
61         int mid = left + (right - left) / 2;
62         if (checkValid(mid, n, s)) {
63             ans = mid;
64             right = mid - 1;
65         } else {
66             left = mid + 1;
67         }
68     }
69
70     cout << ans << "\n";
71 }
72
73 // === 實數二分搜 ===
74 /*
75 需要用 eps 來決定停止狀態 while(r - l > eps)
76 或者是固定迭代次數, 例如 for 100 次後停下
77 */
78
79 // === 找第一個合法 (最小) ===
80 // 答案必在 [l, r] 間
81 while (l < r) {
82     int mid = l + (r - l) / 2;
83
84     if (checkValid(mid))
85         r = mid; // 縮短右邊界, 盡可能找最小
86     else
87         l = mid + 1;
88 }
89 return l; // 回傳是找到的左邊界
90
91 // === 最後一個合法 (最大) ===
92 while (l < r) {
93     int mid = l + (r - l + 1) / 2; // 一定要 +1 才能找最大
94
95     if (checkValid(mid))
96         l = mid; // 縮短左邊界, 盡可能找最大
97     else
98         r = mid - 1;
99 }
100 return l;

```

10.2 Ternary Search

用法: 求單峰/單谷函數極值。浮點數固定跑 100 次迴圈; 整數在 R-L < 2 時改用 for 暴力找極值以避開死迴圈。

時間複雜度: $\mathcal{O}(\log_3(\text{值域}))$

1 // 目標函數: 根據題目需求實作 (此處以尋找單谷函數 (U 型, 二次曲線) 的最小值為例)

```

2 double f(double x) {
3     return x * x - 4 * x + 4; // x^2 - 4x + 4
4 }
5
6 // 浮點數三分搜
7 double ternary_search_double(double L, double R) {
8     // 浮點數一律固定跑 100-200 次 可以保證不會有無窮迴圈與精度問題
9     for (int i = 0; i < 100; ++i) {
10         double m1 = L + (R - L) / 3.0;
11         double m2 = R - (R - L) / 3.0;
12
13         // 若找最小值: 如果 f(m1) > f(m2), 代表真正的底谷一定在 m1 的右邊
14         // (若題目為尋找單峰函數的【最大值】, 將 > 改成 < 即可)
15         if (f(m1) > f(m2)) L = m1;
16         else R = m2;
17     }
18     return f(L);
19 }
20
21 // 整數三分搜 (尋找單谷函數的最小值)
22 long long ternary_search_int(long long L, long long R) {
23     while (R - L > 2) { // 當區間大於 2 時才進行三分搜
24         long long m1 = L + (R - L) / 3;
25         long long m2 = R - (R - L) / 3;
26         if (f(m1) > f(m2)) L = m1;
27         else R = m2;
28     }
29     // 區間縮小到 2 以內時, 直接暴力枚舉 L ~ R 找極值, 完美避開死迴圈
30     long long ans = f(L);
31     for (long long i = L + 1; i <= R; ++i) {
32         ans = min(ans, f(i)); // 若找最大值, 改用 max
33     }
34     return ans;
35 }

```

10.3 Two Pointer 雙指針

用法: 需先自己對陣列做排序

時間複雜度: 大部分 $\mathcal{O}(N)$

```

1 /* === 同向雙指針 ===
2 例如: Subarray sum maximum length
3 給定一個不含負數的 array 與一個定值 x, 要求在 array 中找到總合為 x 的最長
   subarray (不可分割), 輸出最長的長度
4 若有負數, 使用 PrefixSum + Map
5 */
6 int arr[n] = {...};
7 int total = 0, max_len = -1;
8 for (int l = 0, r = 0; r < n; ++r) { // 0(n), r 往右移 n 次
9     total += arr[r];
10    while (total > x && l <= r) {
11        total -= arr[l]; // 扣掉最左邊的值
12        ++l; // l 右移
13    }
14    if (total == x) {

```

```

15        max_len = max(max_len, r-l+1);
16    }
17 }
18
19 /*
20 題意: 給定 target 與 array nums, 求 nums 中 >= target 的 subarray, 最小長度是
   多少
21 題解: r 指針不斷向右跑, 每次都加新數字進來; 隨後判斷目前值是否超過 target, 如果超
   過就不斷地將最左側 (l 指針) 的數字剔除, 最後回傳 len (我們維護的最小長度)
22 */
23 int minSubArrayLen(int target, vector<int>& nums) {
24     int total = 0;
25     int len = 0x3f3f3f3f;
26     for (int l=0, r=0; r<nums.size(); ++r){
27         total += nums[r];
28         while (total >= target){
29             len = min(len, r-l+1);
30             total -= nums[l++];
31         }
32     }
33     return len == 0x3f3f3f3f ? 0 : len;
34 }
35
36 /*
37 題意: 給定一個 array, 請問在固定長度 len 的情況下所能得到的 subarray 的 minimum
38 */
39 int minLenSubArray(int len, vector<int>& nums) {
40     if (nums.size() < len) return 0;
41
42     int cur = 0;
43     for (int i = 0; i < len; ++i) cur += nums[i]; // 一開始就是前 len 個
44     int mini = cur;
45     for (int i = len; i < nums.size(); ++i) {
46         cur += nums[i]; // 加上右邊新進來的元素
47         cur -= nums[i - len]; // 扣掉左邊被擠出去的元素
48         mini = min(mini, cur);
49     }
50
51     return mini;
52 }
53
54 /* === 對向雙指針 ===
55 例如: 2 Sum
56 給定一個 array 與一個定值 x, 找出在 array 中是否存在兩個數的和為 x
57 */
58 int arr[n] = {...};
59 sort(arr, arr+n); // 先做排序, 2 Sum 要有序才能用 Two-Pointers 0(nlogn)
60 bool found = false;
61 int l = 0, r = n-1;
62 while (l < r && !found) { // 當 l == r 即表示不存在
63     int sum = arr[l] + arr[r];
64     if (sum == x) found = true;
65     else if (sum < x) ++l; // 如果 sum 比 x 小, 左邊界往右移
66     else --r; // 反之往左移
67 }
68
69 /*
70 更難的 3 Sum 最佳解也是用 Two-Pointer (0(N^2))
71 給定一個 array 與一個定值 x, 找出在 array 中是否存在三個數的和為 x
72 */
73 int arr[n] = {...}; // 給定陣列
74 sort(arr, arr + n); // 必須先排序 0(nlogn)
75 bool found = false;
76
77 // 先枚舉第一個數字 (從 0~n-3, 最後兩個 (n-2, n-1) 是 l, r)
78 for (int i = 0; i < n - 2; ++i) {
79     // 【優化】如果當前數字與前一個相同, 則跳過, 避免算出重複的組合 (不能直接
       unique 是因為如果數字 x == 3*k, k 被 unique 了就有解變無解)
80     if (i > 0 && arr[i] == arr[i-1]) continue;
81
82     int target = x - arr[i]; // 剩下的兩數之和目標 -> 從這裡開始變回 2 Sum
83     int l = i + 1, r = n - 1; // 雙指針範圍從 i 的下一個到結尾
84
85     while (l < r) {
86         int sum = arr[l] + arr[r];
87
88         if (sum == target) {
89             found = true;
90             break; // 如果只要找「是否存在」就可以直接 break
91
92             /* 若是要找「所有不重複組合」, 可以改成:
93             ans.push_back({arr[i], arr[l], arr[r]});
94             while (l < r && arr[l] == arr[l+1]) ++l; // 跳過重複的左邊界
95             while (l < r && arr[r] == arr[r-1]) --r; // 跳過重複的右邊界
96             ++l; --r; // 繼續尋找下一組
97             */
98         }
99         else if (sum < target) {
100             ++l; // 太小, 左邊界右移
101         } else {
102             --r; // 太大, 右邊界左移
103         }
104     }
105     if (found) break; // 如果已找到, 提早結束外層迴圈
106 }
107
108 // === 快慢雙指針 ===
109 while (fast && fast->next) {
110     slow = slow->next;
111     fast = fast->next->next;
112
113     if (slow == fast)
114         break;
115 }

```

11 MISC

11.1 模板

用法: 競賽用基礎模板

時間複雜度: $NULL$

```
1 #include <bits/stdc++.h>
2
3 #pragma GCC optimize("O3,unroll-loops")
4 #pragma GCC target("avx2,bmi,bmi2,lzcnt,popcnt")
5
6 using namespace std;
7 using ll = long long;
8
9 int main(){
10     ios::sync_with_stdio(0); cin.tie(0); // 或使用 scanf
11     return 0;
12 }
```

11.2 一些概念

用法: 想不到怎麼解或優化的時候可以參考這些想法

時間複雜度: $NULL$

```
1 /*
2 枚舉 (dfs、減少變數降低維度、集合拆半後枚舉、剪枝)
3 貪心 (每次都選局部最佳, 但不能保證局部最佳解是全域最佳解)
4 Random Select(QuickSort)
5
6 Nim Game (賽局理論基礎): 桌上有 N 堆石頭, A1, A2 ... An, 兩人輪流拿。
7 若 A1 ^ A2 ^ A3 ^ ... ^ An != 0, 則先手必勝; 若 XOR 總和為 0, 則後手必勝。
8 */
```

11.3 自訂 Hash Function

用法: 使用 unordered map 或 unordered set 時, 避免被惡意測資卡成 $O(N)$ 時需自訂 Hash function 的情況

時間複雜度: 保證 $O(1)$

```
1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         x += 0x9e3779b97f4a7c15;
4         x = (x ^ (x >> 30)) * 0xbf58476d1ce4e5b9;
5         x = (x ^ (x >> 27)) * 0x94d049bb133111eb;
6         return x ^ (x >> 31);
7     }
8     size_t operator()(uint64_t x) const {
9         static const uint64_t FIXED_RANDOM =
10             chrono::steady_clock::now().time_since_epoch().count();
11         return splitmix64(x + FIXED_RANDOM);
12     };
13 };
14 /*
15 用法:
16 unordered_map<long long, int, custom_hash> safe_map;
17 unordered_set<long long, custom_hash> safe_set;
18 */
```

11.4 C++ 函式庫

用法: 好用的東西可以參考

時間複雜度: $NULL$

```
1 __builtin_popcount(x): 回傳整數 x 轉為二進位後 1 的數量 (若處理 long long 需使用)
2 __builtin_popcountll(x)).
3 __builtin_clz(x): 回傳二進位前導 0 的個數 (Count Leading Zeros).
4 gcd(a, b), lcm(a, b): 求最大公因數 (C++17 引入, 需引入 <numeric>, 舊版編譯器可使用內建的 __gcd(a, b)).
5 string 相關: stoi(s) 將字串轉為 int;
6 to_string(num) 將數字轉為字串;
7 s.substr(pos, len) 擷取子字串.
8
9 #include <algorithm>
10
11 sort(bg, ed) sort(bg, ed, cmp): 預設由小到大排序
12
13 min(a, b) min(list): 取最小值
14 max(a, b) max(list): 取最大值
15
16 min_element(first, last): 找尋連續資料中最小元素
17 max_element(first, last): 找尋連續資料中最大元素
18
19 find(first, last, val): 尋找元素.
20 lower_bound(first, last, val): 尋找第一個 ">=" x 的元素"位置" (如果不存在, 則回傳 last)
21 upper_bound(first, last, val): 尋找第一個 ">" x 的元素"位置", 如果不存在, 則回傳 last
22
23 next_permutation(first, last): 將序列順序轉換成下一個字典序, 如果存在回傳 true, 反之回傳 false
24 prev_permutation(first, last): 將序列順序轉換成上一個字典序, 如果存在回傳 true, 反之回傳 false
25
26 unique(bg, ed): 將相鄰重複元素移至尾端, 回傳新陣列無重複部分尾端的 iterator (或指標)
27 reverse(bg, ed): 反轉區間內的元素順序
28
29 #include <cctype>
30
31 isalnum(int c): 判斷是否為數字或英文
32 isalpha(int c): 判斷是否為英文
33 isdigit(int c): 判斷是否為數字
34
35 islower(int c): 判斷是否為小寫字母
36 isupper(int c): 判斷是否為大寫字母
37
```

```
38 tolower(int c): 將字母轉成小寫字母
39 toupper(int c): 將字母轉成大寫字母
40
41 #include <stack>
42
43 s.push(T a): 插入頂端元素, 複雜度  $O(1)$ 
44 s.pop(): 刪除頂端元素, 複雜度  $O(1)$ 
45 s.top(): 回傳頂端元素, 複雜度  $O(1)$ 
46 s.size(): 回傳元素個數, 複雜度  $O(1)$ 
47 s.empty(): 回傳是否為空, 複雜度  $O(1)$ 
48
49 #include <queue>
50
51 q.push(T a): 插入尾端元素, 複雜度  $O(1)$ 
52 q.pop(): 刪除頂端元素, 複雜度  $O(1)$ 
53 q.front(): 回傳頂端元素, 複雜度  $O(1)$ 
54 q.size(): 回傳元素個數, 複雜度  $O(1)$ 
55 q.empty(): 回傳是否為空, 複雜度  $O(1)$ 
56
57 pq.push(T a): 插入元素 a, 複雜度  $O(\logsize)$ 
58 pq.pop(): 刪除頂端元素, 複雜度  $O(\logsize)$ 
59 pq.top(): 回傳頂端元素, 複雜度  $O(1)$ 
60 pq.size(): 回傳元素個數, 複雜度  $O(1)$ 
61 pq.empty(): 回傳是否為空, 複雜度  $O(1)$ 
62
63 #include <deque>
64
65 dq.push_front(T a), dq.push_back(T a): 插入頂端/尾端元素, 複雜度  $O(1)$ 
66 dq.pop_front(), dq.pop_back(): 刪除頂端/尾端元素, 複雜度  $O(1)$ 
67 dq.front(), dq.back(): 回傳頂端/尾端元素, 複雜度  $O(1)$ 
68 dq.size(): 回傳元素個數, 複雜度  $O(1)$ 
69 dq.empty(): 回傳是否為空, 複雜度  $O(1)$ 
70
71 #include <set>
72
73 s.size(): 回傳元素個數, 複雜度  $O(1)$ 
74 s.empty(): 回傳是否為空, 複雜度  $O(1)$ 
75 s.clear(): 清除元素, 複雜度  $O(size)$ 
76 s.insert(T1 a): 加入元素 a, 複雜度  $O(\logsize)$ 
77 s.erase(iterator first, iterator last): 刪除 [first, last), 若沒有指定 last 則只刪除 first
78 s.erase(T1 a): 刪除鍵值 a, 複雜度  $O(\logsize)$ 
79 s.find(T1 a): 回傳指向鍵值 a 的迭代器, 若不存在則回傳 s.end(), 複雜度  $O(\logsize)$ 
80 s.count(T a): 計算有幾個元素 a
81 s.lower_bound(T1 a): 回傳指向第一個鍵值大於等於 a 的迭代器. 複雜度  $O(\logsize)$ 
82 s.upper_bound(T1 a): 回傳指向第一個鍵值大於 a 的迭代器. 複雜度  $O(\logsize)$ 
83
84 #include <map>
85
86 m.size()
87 m.empty()
88 m.clear()
89 m.count()
90 m.erase(iterator first, iterator last)
91 m.erase(T1 a)
92 m.find(T1 a)
93 m.lower_bound(T1 a)
94 m.upper_bound(T1 a)
```

11.5 對拍器

用法: 有一個想法, 也保證暴力解法全對時, 可以使用對拍器檢查答案

時間複雜度: $NULL$

```
1 // 使用當下時間作為亂數種子, 確保每次執行生成的測資都不同
2 mt19937_64 rng(chrono::steady_clock::now().time_since_epoch().count());
3 /*
4 或 random_device rd; mt19937 rng(rd());
5 mt19937
6 rng(chrono::high_resolution_clock::now().time_since_epoch().count());
7 */
8 // 生成 [L, R] 範圍內的隨機整數
9 long long rnd(long long L, long long R) {
10     return uniform_int_distribution<long long>(L, R)(rng);
11 }
12
13 // 1. 陣列與字串 (Array & String)
14
15 // 生成排列 (Permutation: 1 到 n 不重複)
16 void gen_permutation(int n) {
17     vector<int> p(n);
18     iota(p.begin(), p.end(), 1);
19     shuffle(p.begin(), p.end(), rng);
20     for (int i = 0; i < n; ++i) cout << p[i] << (i == n - 1 ? "" : " ");
21     cout << "\n";
22 }
23
24 // 生成重複值很多的陣列 (distinct_count 控制相異數字的數量)
25 void gen_array_few_distinct(int n, long long L, long long R, int distinct_count) {
26     vector<long long> pool;
27     for (int i = 0; i < distinct_count; ++i) pool.push_back(rnd(L, R));
28     for (int i = 0; i < n; ++i) {
29         cout << pool[rnd(0, distinct_count - 1)] << (i == n - 1 ? "" : " ");
30     }
31     cout << "\n";
32 }
33
34 // 生成已排序或完全反序的陣列
35 void gen_sorted_array(int n, long long L, long long R, bool reverse_sort = false) {
36     vector<long long> a(n);
37     for (int i = 0; i < n; ++i) a[i] = rnd(L, R);
38     if (reverse_sort) sort(a.rbegin(), a.rend());
39     else sort(a.begin(), a.end());
40     for (int i = 0; i < n; ++i) cout << a[i] << (i == n - 1 ? "" : " ");
41     cout << "\n";
42 }
```

```

43
44 // 生成隨機字串
45 void gen_string(int n, string charset = "abcdefghijklmnopqrstuvwxyz") {
46     for (int i = 0; i < n; ++i) {
47         cout << charset[rand(0, charset.size() - 1)];
48     }
49     cout << "\n";
50 }
51
52 // 2. 樹論結構生成器 (Trees)
53
54 // 輔助函式：打亂邊的順序與節點編號，避免特殊的編號順序讓錯誤的 Code 賽中
55 void print_tree(int n, vector<pair<int, int>>& edges, bool weighted = false,
56     long long max_w = 100) {
57     vector<int> p(n + 1);
58     iota(p.begin(), p.end(), 0);
59     shuffle(p.begin() + 1, p.end(), rng); // 打亂 1-n 的編號
60     shuffle(edges.begin(), edges.end(), rng); // 打亂邊的順序
61
62     for (auto& e : edges) {
63         int u = p[e.first], v = p[e.second];
64         if (rnd(0, 1)) swap(u, v); // 隨機交換 u, v 順序
65         cout << u << " " << v;
66         if (weighted) cout << " " << rnd(1, max_w);
67         cout << "\n";
68     }
69
70 // 鏈狀樹 (Chain / Bamboo)：一條線到底 (常卡遞迴深度)
71 void gen_chain(int n, bool weighted = false) {
72     vector<pair<int, int>> edges;
73     for (int i = 2; i <= n; ++i) edges.push_back({i - 1, i});
74     print_tree(n, edges, weighted);
75 }
76
77 // 星狀樹 (Star)：一個中心點連向所有其他點 (常卡度數為 O(N) 的暴力解)
78 void gen_star(int n, bool weighted = false) {
79     vector<pair<int, int>> edges;
80     for (int i = 2; i <= n; ++i) edges.push_back({1, i});
81     print_tree(n, edges, weighted);
82 }
83
84 // 完全二元樹 (Complete Binary Tree)：樹高最低的平衡樹
85 void gen_complete_binary_tree(int n, bool weighted = false) {
86     vector<pair<int, int>> edges;
87     for (int i = 2; i <= n; ++i) edges.push_back({i / 2, i});
88     print_tree(n, edges, weighted);
89 }
90
91 // 毛毛蟲樹 (Caterpillar)：一條主幹，其餘點全部直接連在主幹上
92 void gen_caterpillar(int n, bool weighted = false) {
93     vector<pair<int, int>> edges;
94     int chain_len = max(2LL, rnd(2, n - 1)); // 隨機決定主幹長度
95     for (int i = 2; i <= chain_len; ++i) edges.push_back({i - 1, i});
96     for (int i = chain_len + 1; i <= n; ++i) {
97         int attach_to = rnd(1, chain_len); // 剩下的點隨機接在主幹上
98         edges.push_back({attach_to, i});
99     }
100     print_tree(n, edges, weighted);
101 }
102
103 // 3. 圖論結構生成器 (Graphs)
104
105 // 有向無環圖 (DAG)
106 void gen_dag(int n, int m, bool weighted = false, long long max_w = 100) {
107     // DAG 的特性：只要所有邊都由編號小的連向編號大的，就絕對無環
108     set<pair<int, int>> edges;
109     while (edges.size() < m) {
110         int u = rnd(1, n - 1);
111         int v = rnd(u + 1, n);
112         edges.insert({u, v});
113     }
114
115     // 一樣打亂編號輸出
116     vector<int> p(n + 1);
117     iota(p.begin(), p.end(), 0);
118     shuffle(p.begin() + 1, p.end(), rng);
119
120     vector<pair<int, int>> edge_list(edges.begin(), edges.end());
121     shuffle(edge_list.begin(), edge_list.end(), rng);
122
123     for (auto& e : edge_list) {
124         cout << p[e.first] << " " << p[e.second];
125         if (weighted) cout << " " << rnd(1, max_w);
126         cout << "\n";
127     }
128 }
129
130 // 完全圖 (Complete Graph)：N 個點有 N*(N-1)/2 條邊 (常卡 O(E^2) 演算法)
131 void gen_complete_graph(int n, bool weighted = false, long long max_w = 100) {
132     vector<pair<int, int>> edges;
133     for (int i = 1; i <= n; ++i) {
134         for (int j = i + 1; j <= n; ++j) {
135             edges.push_back({i, j});
136         }
137     }
138     // 不需打亂節點，因為大家互相都有連線，只要打亂邊的順序即可
139     shuffle(edges.begin(), edges.end(), rng);
140     for (auto& e : edges) {
141         if (rnd(0, 1)) cout << e.first << " " << e.second;
142         else cout << e.second << " " << e.first;
143         if (weighted) cout << " " << rnd(1, max_w);
144         cout << "\n";
145     }
146 }
147
148 // 4. 生成隨機區間 [l, r] (Sweep Line, 區間 DP 題型)
149 void gen_intervals(int n, long long max_val) {
150     for (int i = 0; i < n; ++i) {
151         long long l = rnd(1, max_val);
152         long long r = rnd(l, max_val);
153         if (l > r) swap(l, r);
154         cout << l << " " << r << "\n";
155     }
156 }
157
158 // 主程式呼叫範例
159 int main() {
160     ios::sync_with_stdio(false); cin.tie(0);
161
162     // ===== 在這裡調整你的測資 =====
163     // 例如：題目要求輸出 T 筆測資，每筆有 N 個長度為 M 的陣列
164     int T = 1;
165     // cout << T << "\n"; // 如果題目有多筆測資取消註解
166
167     while(T--){
168         int N = 10, M = 15;
169         cout << N << " " << M << "\n";
170
171         // 根據你的需求，取消註解對應的測資生成器
172
173         // [陣列 Edge Cases]
174         // gen_array_few_distinct(N, 1, 100, 2); // 只有 2 種不同數字的陣列
175         // gen_sorted_array(N, 1, 100, true); // 完全反序的陣列
176         // gen_permutation(N); // 1-N 的排列
177
178         // [樹論 Edge Cases]
179         // gen_chain(N, false); // 產生長度為 10 的鏈狀樹 (無權重)
180         // gen_star(N, true); // 產生 10 個點的星狀樹 (帶隨機權重)
181         // gen_caterpillar(N, false); // 產生毛毛蟲樹
182
183         // [圖論 Edge Cases]
184         gen_dag(N, M, true); // 產生 10 點 15 邊的帶權 DAG
185         // gen_complete_graph(N, false); // 產生完全圖
186     }
187
188     return 0;
189 }
190
191 ---
192
193 #, 以下內容直接複製貼上到 bash 即可
194
195 # 1. 編譯所有相關程式 (加上 -O2 模擬正式評測環境)
196 g++ gen.cpp -o gen -O2
197 g++ sol.cpp -o sol -O2
198 g++ brute.cpp -o brute -O2
199
200 echo " 編譯完成，開始對拍..."
201 echo "-----"
202
203 # 2. 進入無限迴圈進行對拍
204 for ((i=1; ; ++i)); do
205
206     # 生成測資並存入 in.txt
207     ./gen > in.txt
208
209     # 將測資餵給正確的暴力解，輸出到 ans.txt
210     ./brute < in.txt > ans.txt
211
212     # 將測資餵給你寫的待測程式，輸出到 out.txt
213     ./sol < in.txt > out.txt
214
215     # 使用 diff 比對兩個輸出檔 (-w 忽略行尾與空白差異)
216     if diff -w out.txt ans.txt > /dev/null; then
217         # 如果沒有差異，印出 AC
218         echo "Test Case $i: AC"
219     else
220         # 如果有差異，停止腳本並報錯
221         echo "Test Case $i: WA! 發現錯誤測資!"
222         break
223     fi
224 done
225
226 echo " 對拍結束。"
227
228 /*
229 執行方法
230
231 1. 準備檔案
232 在同一個資料夾下，準備好四個檔案：
233 sol.cpp：你寫的，會吃 WA 但跑很快的最佳化程式碼。
234 brute.cpp：你寫的，保證絕對正確，但肯定會 TLE 的暴力解程式碼 (例如 O(N!)) 枚舉、
235 O(N^3) 暴力轉移等。
236 gen.cpp：上述的測資生成器。
237 checker.sh：上述的 Bash 腳本。
238
239 2.
240 打開 gen.cpp，修改 main 函式裡面的變數。
241 測資規模 (如 N 或數值範圍) 一定要調小。如果 N 設到 10^5，找到錯誤測資也無法用手算
242 推導。
243 通常設 N <= 10，數值 <= 100 就足夠觸發大部分的 Edge case。
244
245 3. 切換到該資料夾目錄下後，打開 terminal 執行
246
247 4. 賦予腳本執行權限 (只需要做一次)
248 chmod +x checker.sh
249
250 5. 執行對拍
251 ./checker.sh
252
253 當螢幕印出 Test Case X: WA! 發現錯誤測資！時，腳本會自動停止。此時你該資料夾下會
254 擁有：
255 in.txt：引發錯誤的極小測資。(打開它，這就是你一直找不到的 Counter Example)。
256 ans.txt：暴力解跑出來的正確答案。
257 out.txt：你寫的程式跑出來的錯誤答案。
258 */

```

259 文字找碴版：
260 一、資料規模與全域陷阱整數溢位 (**Integer Overflow**)：
261 這是最致命也最常見的失誤。兩個 **int** 相乘或相加是否會超過 **32-bit** 的上限？
262 在 **C++** 中計算時，記得提早轉型，例如寫成 **1LL * a * b**。
263 多筆測資未清空：在 **Codeforces** 或 **LeetCode** 刷題時，如果題目有多筆測試資料，全域變
數、**vector**、**map** 或 **set** 是否在每一輪開始前都有確實執行 **clear()**？
264 未清空會直接污染下一筆測資。
265
266 二、陣列與字串處理極小規模：
267 陣列長度 **N=0** 或 **N=1**。很多迴圈邏輯或雙指標在這種長度下會直接崩潰或發生越界。
268 元素全同：陣列內所有數字都一樣 (例如 **[2, 2, 2, 2]**)。貪心演算法或雙指標有沒有可能
因此陷入無窮迴圈？
269 找不到目標：字串或陣列中完全沒有符合條件的元素。程式是否能正確回傳題目要求的值 (例
如 **-1** 或 **0**)，而不是噴出例外錯誤。
270 索引越界 (**Out of Bounds**)：只要程式碼裡出現 **i-1**、**i+1** 或是二維陣列的八方位搜尋，務
必檢查邊界條件。
271
272 三、圖論與樹圖不連通 (**Disconnected Graph**)：
273 題目如果沒有保證圖是連通的，你的 **DFS** 或 **BFS** 是否能走訪到所有節點？
274 執行 **Kruskal's** 演算法找最小生成樹後，選取的邊數是否確實為 **V-1**？
275 自環與重邊 (**Self-loops & Multiple edges**)：節點自己連向自己，或是節點 **A** 到節點 **B**
之間有兩條權重不同的邊。建圖時是否正確處理了這些冗餘資訊？
276 退化結構：樹退化成一條直線 (**Linked list**)，這會導致遞迴深度達到 **$O(N)$** ，容易引發
Stack Overflow；或是退化成一個中心連向所有點的星狀圖 (**Star graph**)，考驗時間複雜
度。
277 孤島節點：節點數 **V=1** 但邊數 **E=0** 的情況。
278
279 四、進階資料結構與演算法線段樹 (**Segment Tree**) 與 **BIT**：
280 在處理區間查詢 **[L, R]** 時，是否有考慮到 **L > R** 的非法情況？或是 **L = R** 時單點更新是
否正確？
281 掃描線 (**Sweep Line**)：遇到 **X** 座標完全相同的多個點時，事件的處理順序是否正確？(通常
必須先處理「加入區間」的事件，再處理「移除區間」的事件，才不會造成重疊區間斷裂)。
282 **DP** 初始狀態 (**Base Cases**)：動態規劃的狀態轉移前，**dp** 陣列的初始值設定是否合理？求極
小值時有沒有確實初始化為 **INF**？
283
284 五、數值與數學運算除以零與模運算 (**Modulo**)：
285 任何除法或取餘數操作，除數有沒有可能是 **0**？
286 另外，**C++** 的負數取模結果會是負的，遇到減法取模時，必須寫成 **(x % M + M) % M** 來保
證正數。
287 質數與因數邊界：判斷質數時，有沒有正確處理 **0** 和 **1** 既不是質數也不是合數的特例？
288 浮點數誤差：比較兩個浮點數是否相等時，絕對不能直接使用 **==**，請使用 **abs(A - B) <**
1e-9 (或自定義的 **epsilon** 容差值) 來判斷。
289 ***/**